CLAUDE CODE SOURCE STUDY · VOL. 04

Tool, Effect와 Permission 알고리즘

모델의 구조화된 제안을 검증하고 권한을 계산하며 파일·shell 효과로 commit하는 결정적 코드를 읽는다

> 프로젝트의 구조를 외우는 대신, 코드가 어떤 상태와 불변조건을 지키며 결정적인 동작을 보장하는지 이해하기 위한 스터디 보고서다.

이 권에서 얻게 될 통찰

이 권을 마치면 한 번의 Edit가 schema, semantic validation, permission, optimistic concurrency, write와 observation을 거치는 이유를 설명할 수 있다.

Fixed source
Claude Code v2.1.88 복원 snapshot
2ca5ddabfed5f220812ea11f029eda03b21bc4c1
2026-09-20 · Korean study edition

READING BASELINE

OBSERVED

먼저, 지금 보고 있는 시스템의 상태를 고정한다

Tool은 generic schema, 실행, policy, progress, UI projection을 묶은 capability descriptor다. Hook와 sandbox는 이 pipeline의 input·authority·OS capability를 다른 시점에서 바꾼다.

구현에서 직접 확인되는 범위

공통 pipeline은 unknown tool, abort, hook, permission, sandbox routing, call, result를 protocol-valid message로 만든다. 병렬성은 입력별로 계산하고 여러 hook decision은 보수적 precedence로 합성한다.

운영을 말할 때 남겨야 할 경계

파일과 shell 효과는 OS 상태와 경쟁하며 외부 hook process·HTTP endpoint까지 trust boundary에 들어오므로 타입 안전성만으로 보호되지 않는다.

이 보고서가 추측으로 채우지 않는 것

OS별 sandbox runtime의 실제 enforcement 수준, 내부 classifier module, 그리고 production permission와 managed hook policy 값은 현재 snapshot만으로 확정할 수 없다. 이후의 설명은 이 공백을 유지한 채, 실제 코드가 보여 주는 구조와 보장만을 다룬다.

읽는 동안 반복할 질문

이 문장은 코드로 관찰한 사실인가, 구조에서 끌어낸 해석인가, 아니면 개선을 위한 제안인가?

SYSTEM READING MAP

OBSERVED

파일이 아니라 책임과 보장의 연결로 읽는다

이 지도는 번호를 따라 구성요소를 외우게 하지 않는다. 각 영역이 어떤 의미를 소유하고, 그 결과가 다음 영역의 판단 근거가 되며, 마지막에 어떤 외부 보장으로 닫히는지를 문장으로 연결한다.

판단 경계

Describe `Tool registry/schema`

model vocabulary

"model vocabulary"가 확인되어야 다음의 "Parse" 영역이 "unknown→typed"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

판단 경계

Parse `Zod safeParse`

unknown→typed

"unknown→typed"가 확인되어야 다음의 "Validate" 영역이 "현재 상태"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

판단 경계

Validate `semantic checks`

현재 상태

"현재 상태"가 확인되어야 다음의 "Authorize" 영역이 "authority"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

판단 경계

Authorize `rules/hooks/user`

authority

"authority"가 확인되어야 다음의 "Execute" 영역이 "external effect"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

효과·위임

Execute `call`

external effect

"external effect"가 확인되어야 다음의 "Observe" 영역이 "recovery evidence"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

기록·복구

Observe `result/progress/journal`

recovery evidence

이 책임은 앞선 판단을 외부에서 관찰 가능한 결과로 닫는다. 이 지점의 실패 처리가 모호하면 앞의 모든 단계가 성공했더라도 시스템 전체는 정산되지 않은 상태로 남는다.

선을 따라갈 때에는 구현 이름보다 인접한 두 책임 사이에서 전달되는 identity, state, completion을 확인한다. 그 셋 중 하나가 사라지는 지점이 구조적 결함이 나타날 가능성이 가장 높은 경계다.

01 Tool contract와 execution pipeline

generic, schema, factory, executor가 서로 다른 보증을 제공한다.

이 장의 논증은 다음 질문을 차례로 해결한다

코드	효과·위임 Tool generic은 입력과 출력 관계를 보존한다	알고리즘	책임 연결 Generic factory의 default 합성
알고리즘	책임 연결 입력별 concurrency batch partition	알고리즘	상태 소유 병렬 결과의 deterministic context merge
코드	효과·위임 Unknown tool도 error result로 닫힌다	알고리즘	판단 경계 공통 permission-and-call pipeline
구분	효과·위임 Tool pipeline의 질문		

코드 독해의 시작점

`src/Tool.ts` → `src/tools.ts` → `src/services/tools/**`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

SOURCE READING · 1/2

OBSERVED

Tool generic은 입력과 출력 관계를 보존한다

compile-time 관계와 runtime JSON 검증을 혼동하지 않는다.

Tool interface core

src/Tool.ts:362-405

```
362   export type Tool<
363     Input extends AnyObject = AnyObject,
364     Output = unknown,
365     P extends ToolProgressData = ToolProgressData,
366   > = {
367     /**
368      * Optional aliases for backwards compatibility when a tool is renamed.
369      * The tool can be looked up by any of these names in addition to its primary name.
370      */
371     aliases?: string[]
372     /**
373      * One-line capability phrase used by ToolSearch for keyword matching.
374      * Helps the model find this tool via keyword search when it's deferred.
375      * 3–10 words, no trailing period.
376      * Prefer terms not already in the tool name (e.g. 'jupyter' for NotebookEdit).
377      */
378     searchHint?: string
379     call(
380       args: z.infer<Input>,
381       context: ToolUseContext,
382       canUseTool: CanUseToolFn,
383       parentMessage: AssistantMessage,
384       onProgress?: ToolCallProgress<P>,
385     ): Promise<ToolResult<Output>>
386     description(
… 14 lines omitted; use the source link for the complete range …
401     inputsEquivalent?(a: z.infer<Input>, b: z.infer<Input>): boolean
402     isConcurrencySafe(input: z.infer<Input>): boolean
403     isEnabled(): boolean
404     isReadOnly(input: z.infer<Input>): boolean
405     /** Defaults to false. Only set when the tool performs irreversible operations (delete, overwrite, send). */
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	Input은 Zod object schema다.
그 표현이 만든 조건	call은 z.infer<Input>을 받는다.
그 표현이 만든 조건	concurrency/read-only/destructive가 input-dependent다.
그 표현이 만든 조건	description과 presentation도 같은 contract에 있다.

언어와 런타임은 여기까지 보장한다

TypeScript generic은 caller와 implementation의 타입 관계를 유지하지만 네트워크에서 온 unknown 값 자체를 검사하지 않는다.

프로젝트가 이 메커니즘에 맡긴 책임

Zod inputSchema가 runtime boundary를 담당한다.

경계를 잘못 읽을 때 생기는 위험

type assertion만 추가하면 malformed model input이 실행 경계까지 올 수 있다.

이 설명을 확인할 코드 src/Tool.ts:362-405

ALGORITHM A15 · 독자의 질문

Generic factory의 default 합성은 어떤 시스템 문제를 닫는가

수십 개 tool이 같은 실행 계약을 구현할 때 default를 매번 복사하면 작은 차이가 권한과 표시 동작의 차이로 번진다. 반대로 factory가 너무 많은 의미를 숨기면 선언만 읽고 실제 runtime default를 오해한다.

코드가 필요해지는 이유

60개 이상의 tool이 반복 boilerplate 없이 complete Tool contract를 갖게 한다.

OBSERVED Tool.ts의 buildTool이 TOOL_DEFAULTS와 구체 object literal을 합쳐 완성된 Tool을 만든다.

INFERENCE 각 tool은 domain-specific 실행을 소유하고 factory는 공통 default와 타입 관계를 소유한다.

이 절에서 답할 결정 질문

generic factory는 literal type을 보존하면서 어떤 runtime default를 주입하고, spread 순서가 왜 계약의 일부인가?

> ### 이 코드를 읽기 전에 잡을 개념
>
> **구조적 타입과 generic inference.** TypeScript는 선언 이름보다 property shape로 호환성을 판단하며 generic은 입력 literal의 name/schema 관계를 결과까지 보존한다. 그러나 type assertion은 runtime property를 만들지 않는다.
>
> 721–791행에서 defaults와 D의 spread 순서, conditional type, assertion 뒤 실제 object shape를 함께 본다.

이 설명을 확인할 코드 src/Tool.ts:721-791

ALGORITHM A15 · 실제 코드

OBSERVED

Generic factory의 default 합성의 결정점은 어디에 있는가

buildTool은 runtime object spread와 type-level spread를 맞춘다.

입력 표현. 구체 object literal D

보존되는 작업 상태. TOOL_DEFAULTS, 그리고 literal name/schema type

ToolDef/BuiltTool/buildTool

src/Tool.ts:721-791

```
721  export type ToolDef<
722    Input extends AnyObject = AnyObject,
723    Output = unknown,
724    P extends ToolProgressData = ToolProgressData,
725  > = Omit<Tool<Input, Output, P>, DefaultableToolKeys> &
726    Partial<Pick<Tool<Input, Output, P>, DefaultableToolKeys>>
727
728  /**
729   * Type-level spread mirroring `{ ...TOOL_DEFAULTS, ...def }`. For each
730   * defaultable key: if D provides it (required), D's type wins; if D omits
731   * it or has it optional (inherited from Partial<> in the constraint), the
732   * default fills in. All other keys come from D verbatim — preserving arity,
733   * optional presence, and literal types exactly as `satisfies Tool` did.
734   */
… 54 lines omitted; use the source link for the complete range …
789      userFacingName: () => def.name,
790      ...def,
791    } as BuiltTool<D>
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A15 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

구체 object literal D은 처음부터 신뢰된 실행 상태가 아니다. TOOL_DEFAULTS, 그리고 literal name/schema type가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** defaultable key를 optional로 만든 ToolDef를 받는다. 이 결과가 다음 판단 "mapped/conditional type으로 BuiltTool<D>를 계산한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** mapped/conditional type으로 BuiltTool<D>를 계산한다. 이 결과가 다음 판단 "runtime에서 defaults→name fallback→def 순서로 spread한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** runtime에서 defaults→name fallback→def 순서로 spread한다. 이 결과가 다음 판단 "assertion으로 runtime 결과와 계산된 타입을 연결한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** assertion으로 runtime 결과와 계산된 타입을 연결한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

def가 default를 덮어쓴다. literal schema/name 타입을 잃지 않는다. concurrency와 read-only default는 보수적으로 false다.

ALGORITHM A15 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **프로토콜·식별의 질문으로 읽기.** def가 default를 덮어쓴다.
- **구성·변형·성능의 질문으로 읽기.** literal schema/name 타입을 잃지 않는다.
- **프로토콜·식별의 질문으로 읽기.** concurrency와 read-only default는 보수적으로 false다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- spread 순서 변경
- type assertion과 runtime default 불일치
- allow default의 security 의미 오해

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED factory를 통해 default를 중앙화하면서 각 tool의 구체 타입을 보존한다.

RECOMMENDATION compile-only type test와 default behavior contract test를 함께 유지한다.

ALGORITHM A15 · 운영과 현재 상태

작은 변환도 effect와 미확정 경계를 남긴다

등록 시 tool literal을 공통 default와 합쳐 registry가 소비할 object를 만든다.

실패 뒤 남는 상태

spread 순서 역전, runtime 누락, type assertion 과신은 모든 tool에 조용히 전파된다.

OBSERVED	공통 defaults와 generic factory가 구체 tool literal을 완성한다.
CONDITIONAL	tool이 override한 property와 factory merge 순서가 유지될 때 예상 contract가 성립한다.
UNKNOWN	feature-gated 모든 tool이 동일 factory를 거치는지는 이 범위에서 확정되지 않는다.
NEXT READ	default 변경은 단일 함수 수정이 아니라 registry 전체 behavior change로 검토해야 한다.

다시 적용할 질문

1. allow default를 바꾸면 어떤 tool이 명시 override 없이 authority가 달라지는가?
2. compile-time type test가 통과해도 runtime property가 undefined일 수 있는 경로는 무엇인가?

ALGORITHM A16 · 독자의 질문

입력별 concurrency batch partition 은 어떤 시스템 문제를 닫는가

여러 tool call을 모두 직렬화하면 느리고 모두 병렬화하면 같은 resource나 context를 충돌시킬 수 있다. 현재 구현은 안전하다고 분류된 연속 호출만 batch로 묶는다.

코드가 필요해지는 이유

읽기 계열 tool은 병렬화하고 effect 충돌 가능성이 있는 tool은 순서를 보존한다.

OBSERVED — toolOrchestration의 partitionToolCalls가 연속 ToolUseBlock을 Batch[]로 나누고 batch 순서를 보존한다.

INFERENCE — 구체 tool은 자신의 effect를 알지만 orchestrator가 현재 분류를 이용해 schedule을 소유한다.

이 절에서 답할 결정 질문

concurrency-safe라는 분류가 실제 effect의 교환 가능성을 어디까지 표현하며 왜 불확실한 호출은 직렬화하는가?

이 설명을 확인할 코드 src/services/tools/toolOrchestration.ts:84-116

ALGORITHM A16 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/services/tools/toolOrchestration.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01

교환 가능성과 충돌 직렬화

A 뒤 B와 B 뒤 A의 관찰 결과가 같을 때 두 작업은 교환 가능하다. 이름이 read라는 이유만으로 global cache나 context write까지 교환 가능한 것은 아니다.

> **실제 코드로 돌아가면.** 84–116행에서 안전 분류가 무엇을 입력으로 보고 batch 경계를 언제 끊는지 확인한다.

이론은 정답이 아니라 코드에 던질 질문이다

동시성·시간·합의
`commutativity · conflict serializability · conservative scheduling`

stale read, 중복 실행, 취소가 끼어들어도 effect와 최종 상태의 의미가 보존되는가?

ALGORITHM A16 · 실제 코드

OBSERVED

입력별 concurrency batch partition의 결정점은 어디에 있는가

병렬성은 tool 이름이 아니라 parse된 구체 input의 commutativity를 근사한다.

입력 표현. 연속 ToolUseBlock[], 그리고 현재 ToolUseContext

보존되는 작업 상태. Batch[], 그리고 직전 batch의 safety

partitionToolCalls

src/services/tools/toolOrchestration.ts:84-116

```
84    type Batch = { isConcurrencySafe: boolean; blocks: ToolUseBlock[] }
85
86    /**
87     * Partition tool calls into batches where each batch is either:
88     * 1. A single non-read-only tool, or
89     * 2. Multiple consecutive read-only tools
90     */
91    function partitionToolCalls(
92      toolUseMessages: ToolUseBlock[],
93      toolUseContext: ToolUseContext,
94    ): Batch[] {
95      return toolUseMessages.reduce((acc: Batch[], toolUse) => {
96        const tool = findToolByName(toolUseContext.options.tools, toolUse.name)
97        const parsedInput = tool?.inputSchema.safeParse(toolUse.input)
… 16 lines omitted; use the source link for the complete range …
114       return acc
115     }, [])
116   }
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A16 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

연속 ToolUseBlock[], 그리고 현재 ToolUseContext은 처음부터 신뢰된 실행 상태가 아니다. Batch[], 그리고 직전 batch의 safety가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** tool을 registry에서 찾는다. 이 결과가 다음 판단 "schema safeParse를 수행한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** schema safeParse를 수행한다. 이 결과가 다음 판단 "isConcurrencySafe(parsed input)를 호출한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** isConcurrencySafe(parsed input)를 호출한다. 이 결과가 다음 판단 "safe가 연속되면 같은 batch에 합친다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** safe가 연속되면 같은 batch에 합친다. 이 결과가 다음 판단 "unsafe 또는 오류는 단독 serial batch로 만든다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** unsafe 또는 오류는 단독 serial batch로 만든다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

parse 실패는 unsafe다. safety 판정 예외는 unsafe다. unsafe call의 상대 순서를 바꾸지 않는다.

ALGORITHM A16 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **동시성·시간·합의의 질문으로 읽기.** parse 실패는 unsafe다.
- **동시성·시간·합의의 질문으로 읽기.** safety 판정 예외는 unsafe다.
- **동시성·시간·합의의 질문으로 읽기.** unsafe call의 상대 순서를 바꾸지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- read로 분류됐지만 cache/global state를 변경
- 두 safe call의 context modifier 충돌
- 외부 resource contention

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED throughput을 얻되 불확실성은 직렬화하는 fail-conservative 전략이다.

RECOMMENDATION tool effect class와 resource key를 반환해 disjoint-resource 병렬성을 더 정확히 표현한다.

ALGORITHM A16 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

safe 연속 구간은 병렬 batch가 되고 unsafe call은 순서를 가진 단독 batch가 된다.

effect가 시작되고 관찰되는 경계

분류가 놓친 global/cache write는 같은 batch 안에서 race를 만든다.

현재 설계가 선택한 trade-off와 비용

throughput을 얻되 불확실성은 직렬화하는 fail-conservative 전략이다. 이 선택의 계산·운영 비용은 n개 call에 O(n·lookup). tool lookup이 선형이면 O(nK).로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

context modifier 충돌과 외부 resource contention은 tool 이름만으로 보이지 않는다. 이때 함께 대입할 반례는 read로 분류됐지만 cache/global state를 변경, 두 safe call의 context modifier 충돌, 그리고 외부 resource contention이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

tool별 effect class, resource key, batch id, 실제 completion을 기록해야 분류의 정확도를 평가할 수 있다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A16 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	호출 배열을 concurrency-safe 여부에 따라 연속 batch로 partition한다.
CONDITIONAL	분류된 safe tool이 shared state와 외부 resource에서 실제로 교환 가능할 때 determinism이 유지된다.
UNKNOWN	모든 tool의 숨은 effect와 production contention은 source 분류만으로 알 수 없다.
NEXT READ	병렬 범위를 넓히기 전 resource key와 context read/write set을 선언하게 해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. 서로 다른 파일을 읽지만 같은 cache를 갱신하는 두 call은 같은 batch가 가능한가?
2. unsafe call 사이의 두 safe call을 하나의 batch로 건너 묶지 않는 이유는 무엇인가?

이 설명을 확인할 코드 src/services/tools/toolOrchestration.ts:84-116

ALGORITHM A17 · 독자의 질문

병렬 결과의 deterministic context merge은 어떤 시스템 문제를 닫는가

병렬 tool은 progress와 result를 완료되는 즉시 보여 주고 싶지만 shared ToolUseContext를 완료 순서대로 합치면 실행마다 최종 state가 달라질 수 있다.

코드가 필요해지는 이유

progress/result는 병렬로 내보내면서 shared ToolUseContext의 갱신 순서를 안정화한다.

OBSERVED — runTools가 currentContext와 toolUseID별 modifier queue를 소유하며 batch result를 deterministic order로 합친다.

INFERENCE — tool execution은 병렬이지만 context commit은 orchestrator의 단일 merge 경계다.

이 절에서 답할 결정 질문

낮은 latency의 event 순서와 재현 가능한 canonical merge 순서를 어떻게 분리하는가?

이 설명을 확인할 코드 src/services/tools/toolOrchestration.ts:19-81

ALGORITHM A17 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/services/tools/toolOrchestration.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01

two-phase merge와 deterministic reduction

각 worker는 결과와 modifier를 독립적으로 계산하고, shared state 반영은 고정된 순서로 한 번 더 reduce한다. completion order를 commit order로 쓰지 않는 것이 핵심이다.

> **실제 코드로 돌아가면.** 19–81행에서 Promise completion과 modifier application이 서로 다른 순서를 쓰는지 본다.

이론은 정답이 아니라 코드에 던질 질문이다

상태·계산 모델
`deterministic reduction`
단계의 순서나 표현을 바꿔도 같은 상태와 불변조건에 도달하는가?

동시성·시간·합의
`bounded concurrency`
stale read, 중복 실행, 취소가 끼어들어도 effect와 최종 상태의 의미가 보존되는가?

내구성·복구
`two-phase merge`
중단 지점에서 재개했을 때 무엇을 재생하고 무엇을 다시 실행하지 않아야 하는가?

ALGORITHM A17 · 최소 예제

완료 순서와 merge 순서를 분리하면 같은 최종 context를 얻는다

두 tool이 서로 다른 시간에 끝나지만 선언 순서로 modifier를 적용하는 축소 예제다.

설명을 위한 최소 실행 예제

```
results = await Promise.all([run('A', 30), run('B', 5)])
// B progress가 먼저 보여도
for (const result of results) context = result.modify(context)
// merge는 입력 배열 A → B 순서
```

관찰 시점	값과 상태	프로젝트 코드에서의 의미
5ms	B가 먼저 완료	progress latency는 줄지만 아직 canonical context order를 뜻하지 않는다.
30ms	A가 완료	모든 modifier가 준비된다.
merge	A 다음 B로 reduce	반복 실행에서도 같은 context order를 얻는다.

예제를 실제 구현에 다시 연결한다

modifier queue가 toolUseID와 입력 순서를 어떻게 보존하는지 실제 구현에서 확인한다.

ALGORITHM A17 · 실제 코드

OBSERVED

병렬 결과의 deterministic context merge의 결정점은 어디에 있는가

병렬 call 중 context mutation을 즉시 적용하지 않고 tool order대로 지연 합성한다.

입력 표현. concurrency-safe batch, 그리고 MessageUpdateLazy

보존되는 작업 상태. currentContext, 그리고 toolUseID별 modifier queue

runTools batch execution

src/services/tools/toolOrchestration.ts:19-81

```
19    export async function* runTools(
20      toolUseMessages: ToolUseBlock[],
21      assistantMessages: AssistantMessage[],
22      canUseTool: CanUseToolFn,
23      toolUseContext: ToolUseContext,
24    ): AsyncGenerator<MessageUpdate, void> {
25      let currentContext = toolUseContext
26      for (const { isConcurrencySafe, blocks } of partitionToolCalls(
27        toolUseMessages,
28        currentContext,
29      )) {
30        if (isConcurrencySafe) {
31          const queuedContextModifiers: Record<
32            string,
… 46 lines omitted; use the source link for the complete range …
79          }
80        }
81      }
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A17 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

concurrency-safe batch, 그리고 MessageUpdateLazy은 처음부터 신뢰된 실행 상태가 아니다. currentContext, 그리고 toolUseID별 modifier queue가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** 각 tool을 bounded concurrent로 실행한다. 이 결과가 다음 판단 "message는 즉시 yield한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** message는 즉시 yield한다. 이 결과가 다음 판단 "context modifier는 toolUseID별로 모은다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** context modifier는 toolUseID별로 모은다. 이 결과가 다음 판단 "batch가 끝나면 원래 block 순서로 modifier를 적용한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** batch가 끝나면 원래 block 순서로 modifier를 적용한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

동시에 currentContext를 write하지 않는다. merge order는 completion timing이 아니라 input order다. unsafe tool은 이 경로를 사용하지 않는다.

ALGORITHM A17 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **상태·계산 모델의 질문으로 읽기.** 동시에 currentContext를 write하지 않는다.
- **내구성·복구의 질문으로 읽기.** merge order는 completion timing이 아니라 input order다.
- **동시성·시간·합의의 질문으로 읽기.** unsafe tool은 이 경로를 사용하지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- modifier가 실제로 non-commutative
- tool이 context 밖 global을 직접 변경
- progress order를 result order로 오해

이 반례가 나타나면 "알고리즘이 틀렸다"로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 관찰 event의 latency와 canonical context의 determinism을 분리한다.

RECOMMENDATION modifier가 읽고 쓰는 context slice를 선언하고 conflict를 검증한다.

ALGORITHM A17 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

tool은 병렬 실행되고 lazy update가 도착하지만 context modifier는 안정된 batch 순서로 적용된다.

effect가 시작되고 관찰되는 경계

tool이 context 밖 global을 직접 쓰면 two-phase 경계를 우회한다.

현재 설계가 선택한 trade-off와 비용

관찰 event의 latency와 canonical context의 determinism을 분리한다. 이 선택의 계산·운영 비용은 실행 외 merge O(U+M), U는 call 수, M은 modifier 수다.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

non-commutative modifier나 progress order를 result order로 오해하면 UI와 canonical state가 갈라진다. 이때 함께 대입할 반례는 modifier가 실제로 non-commutative, tool이 context 밖 global을 직접 변경, 그리고 progress order를 result order로 오해이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

started/completed/projected/merged 시각과 order를 분리해 기록해야 한다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A17 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	병렬 event projection과 toolUseID별 modifier queue가 분리돼 있다.
CONDITIONAL	modifier가 선언된 context만 변경하고 merge 순서가 안정적일 때 최종 state가 결정적이다.
UNKNOWN	구체 tool의 숨은 global write와 modifier 간 실제 commutativity는 모두 확인되지 않는다.
NEXT READ	modifier마다 read/write context slice를 선언하고 충돌 corpus를 만들어야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. B progress가 먼저 보인 뒤 A가 실패하면 UI와 canonical context는 각각 무엇을 남겨야 하는가?
2. 두 modifier가 같은 field를 쓴다면 fixed order는 determinism만 주는가, 올바른 의미도 보장하는가?

이 설명을 확인할 코드 src/services/tools/toolOrchestration.ts:19-81

SOURCE READING · 1/2

OBSERVED

Unknown tool도 error result로 닫힌다

registry lookup 실패를 throw만 하지 않고 model-visible tool_result로 만든다.

unknown tool path

src/services/tools/toolExecution.ts:337-410

```
337   export async function* runToolUse(
338     toolUse: ToolUseBlock,
339     assistantMessage: AssistantMessage,
340     canUseTool: CanUseToolFn,
341     toolUseContext: ToolUseContext,
342   ): AsyncGenerator<MessageUpdateLazy, void> {
343     const toolName = toolUse.name
344     // First try to find in the available tools (what the model sees)
345     let tool = findToolByName(toolUseContext.options.tools, toolName)
346
347     // If not found, check if it's a deprecated tool being called by alias
348     // (e.g., old transcripts calling "KillShell" which is now an alias for "TaskStop")
349     // Only fall back for tools where the name matches an alias, not the primary name
350     if (!tool) {
351       const fallbackTool = findToolByName(getAllBaseTools(), toolName)
352       // Only use fallback if the tool was found via alias (deprecated name)
353       if (fallbackTool && fallbackTool.aliases?.includes(toolName)) {
354         tool = fallbackTool
355       }
356     }
357     const messageId = assistantMessage.message.id
358     const requestId = assistantMessage.requestId
359     const mcpServerType = getMcpServerType(
360       toolName,
361       toolUseContext.options.mcpClients,
… 44 lines omitted; use the source link for the complete range …
406           toolUseResult: `Error: No such tool available: ${toolName}`,
407           sourceToolAssistantUUID: assistantMessage.uuid,
408         }),
409       }
410       return
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	alias compatibility lookup
그 표현이 만든 조건	analytics용 name sanitization
그 표현이 만든 조건	is_error와 동일 tool_use_id
그 표현이 만든 조건	즉시 return

언어와 런타임은 여기까지 보장한다

protocol completeness는 effect 성공과 별개다.

프로젝트가 이 메커니즘에 맡긴 책임

모델이 다음 단계에서 도구 부재를 관찰하고 대안을 선택할 수 있다.

경계를 잘못 읽을 때 생기는 위험

silent drop은 unresolved tool_use를 남겨 다음 provider request를 깨뜨린다.

이 설명을 확인할 코드 src/services/tools/toolExecution.ts:337-410

ALGORITHM A18 · 독자의 질문

공통 permission-and-call pipeline 은 어떤 시스템 문제를 닫는가

모델이 만든 JSON은 아직 실행 권한이 없는 외부 입력이다. schema parsing, hook 변형, permission, 실행이 다른 순서로 배치되면 승인한 값과 실행한 값이 달라질 수 있다.

코드가 필요해지는 이유

모델이 만든 unknown JSON을 현재 workspace에서 실행 가능한 승인된 input으로 바꾼다.

OBSERVED checkPermissionsAndCallTool이 parsed/updated input, permission result, progress, duration을 순서대로 소유한다.

INFERENCE tool 구현은 실제 effect를 수행하고 공통 pipeline은 validation과 authority ordering을 소유한다.

이 절에서 답할 결정 질문

unknown input이 승인된 executable input으로 바뀌는 각 단계에서 representation과 authority는 어떻게 변하는가?

이 설명을 확인할 코드 src/services/tools/toolExecution.ts:599-720

ALGORITHM A18 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/services/tools/toolExecution.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01 reference monitor와 TOCTOU

모든 effect 요청은 우회할 수 없는 공통 검사점을 지나야 한다. 검사한 값과 사용하는 값 사이에서 mutation이 일어나면 time-of-check와 time-of-use가 갈라진다.

> **실제 코드로 돌아가면.** 599–720행에서 parse·hook·permission·call 사이에 input identity가 언제 바뀌는지 본다.

이론은 정답이 아니라 코드에 던질 질문이다

권한·신뢰 경계
`reference monitor · policy enforcement point`
누가 어떤 근거로 effect를 허용하며, 더 약한 권한으로만 위임된다는 사실을 어디서 확인하는가?

상태·계산 모델
`pipeline`
단계의 순서나 표현을 바꿔도 같은 상태와 불변조건에 도달하는가?

ALGORITHM A18 · 최소 예제

승인 뒤 input이 바뀌면 같은 tool call도 다른 effect가 된다

검사한 path와 실행한 path가 달라지는 최소 반례다.

설명을 위한 최소 실행 예제

```
let input = { path: '/safe/a' }
permit(input)
input.path = '/secret/b'
callTool(input) // 허용 판단과 실행 대상이 다름
```

관찰 시점	값과 상태	프로젝트 코드에서의 의미
parse	/safe/a를 구조화	아직 authority 없음
permission	/safe/a를 허용	결정은 이 representation에만 유효
mutation	/secret/b로 변경	기존 decision을 재사용하면 TOCTOU 발생

예제를 실제 구현에 다시 연결한다

실제 pipeline에서 hook updatedInput이 permission 전인지 후인지와 digest가 유지되는지 추적한다.

ALGORITHM A18 · 실제 코드

OBSERVED

공통 permission-and-call pipeline의 결정점은 어디에 있는가

schema가 맞는 입력을 곧바로 실행하지 않고 hook·validation·authority·call 순서를 통과시킨다.

입력 표현. Tool, toolUseID, raw input, ToolUseContext, 그리고 canUseTool

보존되는 작업 상태. parsed/updated input, permission result, progress, 그리고 duration

checkPermissionsAndCallTool 앞부분

src/services/tools/toolExecution.ts:599-720

```
599  async function checkPermissionsAndCallTool(
600    tool: Tool,
601    toolUseID: string,
602    input: { [key: string]: boolean | string | number },
603    toolUseContext: ToolUseContext,
604    canUseTool: CanUseToolFn,
605    assistantMessage: AssistantMessage,
606    messageId: string,
607    requestId: string | undefined,
608    mcpServerType: McpServerType,
609    mcpServerBaseUrl: ReturnType<typeof getLoggingSafeMcpBaseUrl>,
610    onToolProgress: (
611      progress: ToolProgress<ToolProgressData> | ProgressMessage<HookProgress>,
612    ) => void,
… 105 lines omitted; use the source link for the complete range …
718        {
719          message: createUserMessage({
720            content: [
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A18 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

Tool, toolUseID, raw input, ToolUseContext, 그리고 canUseTool은 처음부터 신뢰된 실행 상태가 아니다. parsed/updated input, permission result, progress, 그리고 duration가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** runtime schema와 validateInput을 적용한다. 이 결과가 다음 판단 "pre-tool hook가 입력/결정을 수정할 수 있다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** pre-tool hook가 입력/결정을 수정할 수 있다. 이 결과가 다음 판단 "tool-specific permission을 계산한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** tool-specific permission을 계산한다. 이 결과가 다음 판단 "필요하면 human/host canUseTool을 호출한다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** 필요하면 human/host canUseTool을 호출한다. 이 결과가 다음 판단 "승인된 최종 input으로 call한다."의 전제가 된다.
효과 조정	**표현과 선택지를 좁힌다** 승인된 최종 input으로 call한다. 이 결과가 다음 판단 "success/failure hook와 result projection을 수행한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** success/failure hook와 result projection을 수행한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

사용자가 승인한 input과 call input의 identity를 추적한다. deny/abort에서도 terminal tool_result를 만든다. validation과 authority를 같은 Boolean으로 축약하지 않는다.

ALGORITHM A18 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **권한·신뢰 경계의 질문으로 읽기.** 사용자가 승인한 input과 call input의 identity를 추적한다.
- **상태·계산 모델의 질문으로 읽기.** deny/abort에서도 terminal tool_result를 만든다.
- **권한·신뢰 경계의 질문으로 읽기.** validation과 authority를 같은 Boolean으로 축약하지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- 승인 뒤 input mutation
- hook timeout 정책 불명확
- validation과 permission 사이 TOCTOU

이 반례가 나타나면 "알고리즘이 틀렸다"로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 도구 구현마다 반복하지 않고 ordering-sensitive control을 중앙화한다.

RECOMMENDATION 각 transformation에 input digest와 provenance를 기록하는 decision trace를 만든다.

ALGORITHM A18 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

raw input을 parse/normalize하고 hook와 permission을 거쳐 tool을 호출하며 progress/result를 투영한다.

effect가 시작되고 관찰되는 경계

callTool 진입이 외부 effect 경계이며 그 직전 input과 decision이 같은 identity여야 한다.

현재 설계가 선택한 trade-off와 비용

도구 구현마다 반복하지 않고 ordering-sensitive control을 중앙화한다. 이 선택의 계산·운영 비용은 각 단계 비용의 합. hook와 classifier는 외부 I/O를 포함할 수 있다.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

승인 후 mutation, timeout policy 모호성, executor 우회는 reference monitor를 깨뜨린다. 이때 함께 대입할 반례는 승인 뒤 input mutation, hook timeout 정책 불명확, 그리고 validation과 permission 사이 TOCTOU이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

각 transformation의 before/after digest와 decision source를 남겨야 한다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A18 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	공통 permission-and-call pipeline과 ordering-sensitive 단계가 있다.
CONDITIONAL	모든 tool executor가 이 경계를 통과하고 최종 input에 permission이 적용될 때 authority가 맞는다.
UNKNOWN	별도 executor와 feature variant의 우회 여부는 선택 범위만으로 닫히지 않는다.
NEXT READ	새 transformation 추가 시 permission 재평가 조건과 provenance trace를 먼저 정의해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. hook가 path를 바꾼다면 기존 permission result를 재사용할 수 있는가?
2. schema parsing이 default 값을 추가하면 사용자가 승인 화면에서 그 값을 볼 수 있는가?

이 설명을 확인할 코드 src/services/tools/toolExecution.ts:599-720

DISTINCTION TABLE

OBSERVED

Tool pipeline의 질문

각 단계가 답하는 질문이 다르다.

단계	질문	실패 의미
Schema	형식이 맞는가	unknown→typed 실패
Semantic validation	현재 상태에서 가능한가	stale/invalid target
Permission	이 subject가 해도 되는가	authority 부재
Sandbox	피해를 어디까지 제한하는가	containment 실패
Observation	무엇이 일어났는가	복구/설명 실패

이 표를 읽는 관점

한 단계가 다른 단계를 대체하지 않는다.

02 도구 목록은 제품의 effect surface다

FileEdit 몇 개만 깊게 읽고 tool system 전체를 이해했다고 말할 수 없다. 반대로 40여 개 이름을 나열하는 것도 충분하지 않다. registry가 capability를 포함·제외하는 조건과 각 capability의 effect class를 함께 읽어야 한다.

이 장의 논증은 다음 질문을 차례로 해결한다

코드	효과·위임 getAllBaseTools는 정적 배열이 아니라 build·environment·product mode의 합성 결과다	구분	효과·위임 Effect class가 달라지면 필요한 보장도 달라진다
사례	효과·위임 새 tool을 registry에 더할 때의 완전한 영향 경로	해석	효과·위임 모든 leaf tool을 같은 깊이로 설명하지 않는 이유

코드 독해의 시작점

`src/tools.ts` → `src/Tool.ts` → `src/tools/**`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

SOURCE READING · 1/2

OBSERVED

getAllBaseTools는 정적 배열이 아니라 build·environment·product mode의 합성 결과다

모델이 볼 수 있는 capability universe는 import 목록, compile-time feature, USER_TYPE, runtime gate, embedded search, task/team/worktree mode를 조합해 만든다.

base import와 compile/environment-gated capability

src/tools.ts:1-150

```
1     // biome-ignore-all assist/source/organizeImports: ANT-ONLY import markers must not be reordered
2     import { toolMatchesName, type Tool, type Tools } from './Tool.js'
3     import { AgentTool } from './tools/AgentTool/AgentTool.js'
4     import { SkillTool } from './tools/SkillTool/SkillTool.js'
5     import { BashTool } from './tools/BashTool/BashTool.js'
6     import { FileEditTool } from './tools/FileEditTool/FileEditTool.js'
7     import { FileReadTool } from './tools/FileReadTool/FileReadTool.js'
8     import { FileWriteTool } from './tools/FileWriteTool/FileWriteTool.js'
9     import { GlobTool } from './tools/GlobTool/GlobTool.js'
10    import { NotebookEditTool } from './tools/NotebookEditTool/NotebookEditTool.js'
11    import { WebFetchTool } from './tools/WebFetchTool/WebFetchTool.js'
12    import { TaskStopTool } from './tools/TaskStopTool/TaskStopTool.js'
13    import { BriefTool } from './tools/BriefTool/BriefTool.js'
14    // Dead code elimination: conditional import for ant-only tools
15    /* eslint-disable custom-rules/no-process-env-top-level, @typescript-eslint/no-require-imports */
16    const REPLTool =
17      process.env.USER_TYPE === 'ant'
18        ? require('./tools/REPLTool/REPLTool.js').REPLTool
19        : null
20    const SuggestBackgroundPRTool =
21      process.env.USER_TYPE === 'ant'
22        ? require('./tools/SuggestBackgroundPRTool/SuggestBackgroundPRTool.js')
23            .SuggestBackgroundPRTool
24        : null
25    const SleepTool =

… 120 lines omitted; use the source link for the complete range …

146     isReplModeEnabled,
147   } from './tools/REPLTool/constants.js'
148   export { REPL_ONLY_TOOLS }
149   /* eslint-disable @typescript-eslint/no-require-imports */
150   const getPowerShellTool = () => {
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	REPL·KAIROS·browser·workflow·snip 같은 tool은 feature 또는 user type에 따라 module 자체가 없을 수 있다.
그 표현이 만든 조건	Team/SendMessage는 circular dependency를 피하려고 lazy require한다.
그 표현이 만든 조건	PowerShell·LSP·worktree·task tool은 runtime environment와 product mode를 확인한다.
그 표현이 만든 조건	MCP resource와 ToolSearch capability는 base list 뒤에 더해진다.

언어와 런타임은 여기까지 보장한다

registry는 단순 dependency list가 아니라 이 process에서 존재 가능한 capability의 composition root다. compile-time false는 module graph에서 제거될 수 있고, runtime isEnabled는 포함된 descriptor가 현재 사용 가능한지를 다시 판단한다.

프로젝트가 이 메커니즘에 맡긴 책임

getAllBaseTools가 가능한 universe를 만들고 preset·permission filtering·request-time tool search가 실제 provider request에 노출할 subset을 만든다. “파일이 있다”와 “모델이 호출할 수 있다”는 다른 상태다.

경계를 잘못 읽을 때 생기는 위험

README의 tool 개수나 import 수만으로 실제 surface를 확정하면 feature-gated missing module과 environment-dependent tool을 잘못 포함한다. system prompt cache와 registry order가 결합돼 있어 순서 변경도 비용·cache 의미를 바꿀 수 있다.

이 설명을 확인할 코드 src/tools.ts:1-150 src/tools.ts:151-260

DISTINCTION TABLE

OBSERVED

Effect class가 달라지면 필요한 보장도 달라진다

모든 tool에 같은 permission 카드 하나를 붙이는 대신 실제로 바꾸는 world와 recovery evidence를 기준으로 분류한다.

Effect class	대표 capability	필수 질문
Pure/control	plan mode, question, config projection	외부 effect가 정말 없는가, session control은 durable한가?
Workspace read	Read, Glob, Grep, LSP query	어떤 snapshot을 읽었고 다음 write precondition으로 쓰이는가?
Workspace write	Edit, Write, NotebookEdit	stale check·atomicity·partial write·replay disposition은 무엇인가?
Process execution	Bash, PowerShell, background task	shell grammar·cwd·env·sandbox·exit/cancel을 누가 소유하는가?
Network/retrieval	WebFetch, WebSearch, MCP	URL/DNS/proxy/auth/data egress와 remote schema를 어떻게 제한하는가?
Coordination/control plane	Agent, Task, Team, SendMessage, Worktree	child authority·mailbox·durable task·workspace isolation이 어떻게 정산되는가?
Dynamic extension	Skill, ToolSearch, plugin-supplied MCP	provenance·late discovery·name collision·activation을 어디서 검증하는가?

이 표를 읽는 관점

분류는 영구 label이 아니다. input에 따라 read-only가 write나 process effect로 바뀌는 tool은 Tool의 input-dependent isReadOnly·isConcurrencySafe와 실제 call을 함께 봐야 한다.

SELECTIVE EXECUTION TRACE

OBSERVED

새 tool을 registry에 더 할 때의 완전한 영향 경로

capability descriptor 하나를 추가하는 일은 실행 함수 하나를 등록하는 것보다 넓다.

책임 연결

Existence

build variant와 environment에서 module이 포함되는지 결정한다.

이때 남은 상태와 판단이 "Discovery"에서 사용할 전제가 된다.

책임 연결

Discovery

name·description·schema·prompt가 model vocabulary와 cache prefix에 들어간다.

이때 남은 상태와 판단이 "Admission"에서 사용할 전제가 된다.

책임 연결

Admission

preset, isEnabled, permission deny filter, tool search가 request subset을 정한다.

이때 남은 상태와 판단이 "Authority"에서 사용할 전제가 된다.

책임 연결

Authority

parsed input의 effect class에 맞는 validation·hook·permission·sandbox를 통과한다.

이때 남은 상태와 판단이 "Execution"에서 사용할 전제가 된다.

책임 연결

Execution

concurrency partition과 cancellation 아래에서 effect를 commit한다.

이때 남은 상태와 판단이 "Projection"에서 사용할 전제가 된다.

관찰·정산

Projection

progress·result·UI·SDK·transcript representation을 만든다.

이때 남은 상태와 판단이 "Recovery"에서 사용할 전제가 된다.

기록·복구

Recovery

effect는 일어났지만 result가 유실된 경우 replay/inspect/cleanup disposition을 제공한다.

이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

tool file과 registry import가 있다는 사실만으로 discovery, authority, durability가 닫히지 않는다.

어디에서 설명이 틀어지는가

한 단계의 owner나 terminal state가 정의되지 않으면 새 capability는 존재하더라도 운영 가능한 contract가 아니다.

이 설명을 확인할 코드 src/tools.ts:151-260 src/Tool.ts:362-405

CONCEPT IN CONTEXT

INFERENCE

모든 leaf tool을 같은 깊이로 설명하지 않는 이유

전체 coverage는 모든 capability family를 owner·activation·effect·recovery 축으로 지도화하고, 위험과 구조를 대표하는 결정적 경로를 깊게 읽는 방식으로 달성한다.

코드에서 읽어낼 수 있는 단서

출발 단서	FileRead/FileEdit는 observation과 optimistic write의 관계를 보여 준다.
연결되는 단서	Bash는 외부 언어와 process effect를 보여 준다.
연결되는 단서	MCP는 external schema와 transport trust boundary를 보여 준다.
연결되는 단서	Agent/Task/Team/Worktree는 nested runtime과 coordination state를 보여 준다.
연결되는 단서	나머지 leaf는 같은 effect class의 공통 contract와 자신만의 예외를 scope ledger에서 추적한다.

이 단서들을 하나의 설계로 묶으면

184개 tool source 파일을 한 문장씩 요약하면 분량은 늘지만 독자는 새 tool의 failure와 변경 영향을 예측할 수 없다. 대표 경로의 상세 분석과 전체 registry/effect taxonomy를 결합해야 breadth와 depth가 동시에 생긴다.

시스템 이해에 남는 결론

새 tool family가 기존 effect class로 설명되지 않거나 별도 authority·durability model을 도입하면 그때 독립 학습 단위로 승격해야 한다.

이 설명을 확인할 코드 src/tools.ts:151-260

03 권한과 shell 의미를 계산한다

permission은 source precedence, behavior algebra, tool-specific semantics가 겹치는 의사결정이다.

이 장의 논증은 다음 질문을 차례로 해결한다

코드	판단 경계 Permission result는 algebraic data type이다	알고리즘	책임 연결 Rule source aggregation과 behavior precedence
알고리즘	효과·위임 Tool-level deny/ask short circuit	코드	진입 경계 Shell command는 문자열이 아니라 외부 언어다
알고리즘	책임 연결 Compound command decision 합성	구분	판단 경계 Permission과 sandbox
해석	책임 연결 Path는 문자열이 아니라 시간에 따라 변하는 resource identity다		

코드 독해의 시작점

`src/types/permissions.ts` → `src/utils/permissions/**` → `src/tools/BashTool/**`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

SOURCE READING · 1/2

OBSERVED

Permission result는 algebraic data type이다

allow·ask·deny·passthrough는 점수 구간이 아니라 서로 다른 후속 protocol이다.

permission decision union

src/types/permissions.ts:171-260

```ts
  /**
   * Result when permission is granted
   */
  export type PermissionAllowDecision<
    Input extends { [key: string]: unknown } = { [key: string]: unknown },
  > = {
    behavior: 'allow'
    updatedInput?: Input
    userModified?: boolean
    decisionReason?: PermissionDecisionReason
    toolUseID?: string
    acceptFeedback?: string
    contentBlocks?: ContentBlockParam[]
  }

  /**
   * Metadata for a pending classifier check that will run asynchronously.
   * Used to enable non-blocking allow classifier evaluation.
   */
  export type PendingClassifierCheck = {
    command: string
    cwd: string
    descriptions: string[]
  }

… 60 lines omitted; use the source link for the complete range …

        behavior: 'passthrough'
        message: string
        decisionReason?: PermissionDecision<Input>['decisionReason']
        suggestions?: PermissionUpdate[]
        blockedPath?: string
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	allow는 updatedInput을 가질 수 있다.
그 표현이 만든 조건	ask는 suggestion과 pending classifier를 가질 수 있다.
그 표현이 만든 조건	deny는 reason이 필수다.
그 표현이 만든 조건	passthrough는 다음 policy layer로 넘긴다.

언어와 런타임은 여기까지 보장한다

discriminated union은 variant별 required payload와 exhaustive handling을 가능하게 한다.

프로젝트가 이 메커니즘에 맡긴 책임

authority 판단과 사용자 interaction을 Boolean 하나로 줄이지 않는다.

경계를 잘못 읽을 때 생기는 위험

새 behavior를 추가하면 SDK schema, UI, tool pipeline이 모두 변한다.

이 설명을 확인할 코드 src/types/permissions.ts:171-260

ALGORITHM A19 · 독자의 질문

Rule source aggregation과 behavior precedence은 어떤 시스템 문제를 닫는가

허용 규칙은 user·project·local·flag·managed policy 등 여러 source에 있고 같은 command에 서로 다른 결론을 낼 수 있다. 최종 값만 남기면 왜 그 결정이 났는지 설명할 수 없다.

코드가 필요해지는 이유

user/project/local/flag/policy/CLI/command/session rule을 provenance와 함께 effective decision에 사용한다.

OBSERVED permissions module이 source-tagged rule 배열을 구성해 downstream matcher에 넘긴다.

INFERENCE 각 설정 source가 rule을 소유하고 aggregation이 effective evaluation order를 소유한다.

이 절에서 답할 결정 질문

source order와 behavior precedence는 어떤 후보를 선택하며 provenance를 어디까지 보존하는가?

이 설명을 확인할 코드 src/utils/permissions/permissions.ts:109-131 src/utils/settings/constants.ts:3-22

ALGORITHM A19 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/utils/permissions/permissions.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01
policy combining과 provenance

여러 decision을 합칠 때 first-match, deny-overrides, specificity 같은 알고리즘이 결과를 바꾼다. provenance는 winner뿐 아니라 검토된 후보를 설명하는 근거다.

> **실제 코드로 돌아가면.** 109–131행의 배열 순서가 단순 표시 순서인지 matcher precedence인지 downstream과 함께 본다.

이론은 정답이 아니라 코드에 던질 질문이다

권한·신뢰 경계
`policy combining algorithm · provenance · deny/permit precedence`
누가 어떤 근거로 effect를 허용하며, 더 약한 권한으로만 위임된다는 사실을 어디서 확인하는가?

ALGORITHM A19 · 실제 코드

OBSERVED

Rule source aggregation과 behavior precedence의 결정점은 어디에 있는가

설정 source 순서와 allow/deny/ask 평가 순서는 같은 개념이 아니다.

입력 표현. ToolPermissionContext의 behavior별 rules

보존되는 작업 상태. source-tagged PermissionRule[]

source-ordered allow rules

src/utils/permissions/permissions.ts:109-131

```
109  const PERMISSION_RULE_SOURCES = [
110    ...SETTING_SOURCES,
111    'cliArg',
112    'command',
113    'session',
114  ] as const satisfies readonly PermissionRuleSource[]
115
116  export function permissionRuleSourceDisplayString(
117    source: PermissionRuleSource,
118  ): string {
119    return getSettingSourceDisplayNameLowercase(source)
120  }
121
122  export function getAllowRules(
… 6 lines omitted; use the source link for the complete range …
129        ruleValue: permissionRuleValueFromString(ruleString),
130      })),
131    )
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A19 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

ToolPermissionContext의 behavior별 rules은 처음부터 신뢰된 실행 상태가 아니다. source-tagged PermissionRule[]가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** SETTING_SOURCES 순서에 CLI/command/session을 덧붙인다. 이 결과가 다음 판단 "behavior별 string을 parsed rule로 바꾼다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** behavior별 string을 parsed rule로 바꾼다. 이 결과가 다음 판단 "tool-specific matcher로 deny·ask·allow 후보를 찾는다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** tool-specific matcher로 deny·ask·allow 후보를 찾는다. 이 결과가 다음 판단 "mode와 hooks 등 상위 decision path가 결과를 조합한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** mode와 hooks 등 상위 decision path가 결과를 조합한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

rule에는 source가 남는다. managed policy를 단순한 session override로 보지 않는다. behavior precedence와 setting override order를 혼동하지 않는다.

ALGORITHM A19 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **권한·신뢰 경계의 질문으로 읽기.** rule에는 source가 남는다.
- **권한·신뢰 경계의 질문으로 읽기.** managed policy를 단순한 session override로 보지 않는다.
- **권한·신뢰 경계의 질문으로 읽기.** behavior precedence와 setting override order를 혼동하지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- 첫 match가 intended specificity와 다름
- source order 그림의 과도한 단순화
- rule parser와 shell parser 불일치

이 반례가 나타나면 "알고리즘이 틀렸다"로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 값뿐 아니라 "왜"를 설명하기 위해 provenance를 decision에 유지한다.

RECOMMENDATION decision trace에 검토한 후보와 선택/배제 이유를 남긴다.

ALGORITHM A19 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

source별 rule을 provenance와 함께 모아 permission decision 입력으로 만든다.

effect가 시작되고 관찰되는 경계

선택된 rule은 이후 tool effect를 허용·질문·거부하는 authority가 된다.

현재 설계가 선택한 trade-off와 비용

값뿐 아니라 "왜"를 설명하기 위해 provenance를 decision에 유지한다. 이 선택의 계산·운영 비용은 rule 수 R과 matcher 비용에 비례.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

parser 차이, broad rule, 그림으로 단순화한 source order는 의도와 다른 match를 만든다. 이때 함께 대입할 반례는 첫 match가 intended specificity와 다름, source order 그림의 과도한 단순화, 그리고 rule parser와 shell parser 불일치이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

candidate, match reason, winner, excluded reason을 남겨야 정책을 설명할 수 있다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A19 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	source-tagged allow/ask/deny rules를 일정 순서로 모은다.
CONDITIONAL	matcher의 specificity와 behavior precedence가 aggregation 순서와 합치될 때 문서화된 의미가 성립한다.
UNKNOWN	managed policy의 실제 값과 배포별 source 활성화는 알 수 없다.
NEXT READ	precedence 변경 전 representative conflict corpus와 decision trace를 고정해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. project allow와 managed deny가 동시에 맞으면 어느 결론이 나고 후보 목록에 무엇이 남는가?
2. 같은 source 안의 broad rule과 specific rule은 source order만으로 해결되는가?

이 설명을 확인할 코드 src/utils/permissions/permissions.ts:109-131 src/utils/settings/constants.ts:3-22

ALGORITHM A20 · 독자의 질문

Tool-level deny/ask short circuit은 어떤 시스템 문제를 닫는가

도구 전체나 MCP namespace를 금지하는 coarse rule이 있으면 복잡한 input parsing 전에 빠르게 멈출 수 있다. 하지만 alias와 namespace가 충돌하면 싸게 내린 결정이 잘못된 identity에 적용된다.

코드가 필요해지는 이유

특정 tool 또는 MCP namespace 전체에 대한 정책을 input parsing과 별도로 적용한다.

OBSERVED — tool permission path가 AppState context와 tool/MCP identity로 deny/ask rule을 먼저 찾는다.

INFERENCE — coarse gate는 뒤의 command/path 분석을 대체하지 않고 명백한 상위 정책을 먼저 적용한다.

이 절에서 답할 결정 질문

tool-level short circuit은 어떤 identity를 신뢰하고, fine-grained 검사에 무엇을 남기는가?

> ### 이 코드를 읽기 전에 잡을 개념
>
> **short-circuit authorization.** 상위 정책이 terminal decision을 내리면 비싼 세부 분석을 생략한다. 그 전제는 identity normalization과 namespace가 충돌 없이 안정적이라는 것이다.
>
> 1070–1120행에서 deny·ask가 언제 return되고 allow가 어느 fine-grained 경로로 이어지는지 본다.

이 설명을 확인할 코드 src/utils/permissions/permissions.ts:1070-1120

ALGORITHM A20 · 실제 코드

OBSERVED

Tool-level deny/ask short circuit의 결정점은 어디에 있는가

도구 전체 rule은 구체 input permission보다 앞서 빠른 terminal decision을 만든다.

입력 표현. AppState permission context, 그리고 Tool name/MCP identity

보존되는 작업 상태. matched deny/ask rule

tool permission decision path

src/utils/permissions/permissions.ts:1070-1120

```
1070   */
1071  export async function checkRuleBasedPermissions(
1072    tool: Tool,
1073    input: { [key: string]: unknown },
1074    context: ToolUseContext,
1075  ): Promise<PermissionAskDecision | PermissionDenyDecision | null> {
1076    const appState = context.getAppState()
1077
1078    // 1a. Entire tool is denied by rule
1079    const denyRule = getDenyRuleForTool(appState.toolPermissionContext, tool)
1080    if (denyRule) {
1081      return {
1082        behavior: 'deny',
1083        decisionReason: {
… 34 lines omitted; use the source link for the complete range …
1118    try {
1119      const parsedInput = tool.inputSchema.parse(input)
1120      toolPermissionResult = await tool.checkPermissions(parsedInput, context)
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A20 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

AppState permission context, 그리고 Tool name/MCP identity은 처음부터 신뢰된 실행 상태가 아니다. matched deny/ask rule가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** deny rule을 먼저 찾는다. 이 결과가 다음 판단 "있으면 provenance가 있는 deny를 반환한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** 있으면 provenance가 있는 deny를 반환한다. 이 결과가 다음 판단 "ask rule을 찾는다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** ask rule을 찾는다. 이 결과가 다음 판단 "있으면 user-facing request와 suggestion을 만든다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** 있으면 user-facing request와 suggestion을 만든다. 이 결과가 다음 판단 "없으면 passthrough로 tool-specific 검사에 넘긴다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** 없으면 passthrough로 tool-specific 검사에 넘긴다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

deny는 하위 allow가 조용히 덮지 않는다. MCP display name collision을 fully-qualified identity로 구분한다. passthrough를 allow로 오해하지 않는다.

ALGORITHM A20 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **상태·계산 모델의 질문으로 읽기.** deny는 하위 allow가 조용히 덮지 않는다.
- **프로토콜·식별의 질문으로 읽기.** MCP display name collision을 fully-qualified identity로 구분한다.
- **권한·신뢰 경계의 질문으로 읽기.** passthrough를 allow로 오해하지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- alias와 namespace collision
- broad rule이 intended tool을 과잉 차단
- passthrough consumer 누락

이 반례가 나타나면 "알고리즘이 틀렸다"로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED cheap coarse policy를 먼저 적용하고 command/path 의미 분석은 뒤에 둔다.

RECOMMENDATION compiled rule index와 specificity test를 추가하되 기존 precedence를 보존한다.

ALGORITHM A20 · 운영과 현재 상태

작은 변환도 effect와 미확정 경계를 남긴다

tool identity로 coarse rule을 조회해 deny/ask면 즉시 반환하고 아니면 세부 검사로 넘긴다.

실패 뒤 남는 상태

alias collision과 과도한 broad rule은 intended tool을 잘못 차단하거나 통과시킨다.

OBSERVED	tool/MCP identity에 대한 coarse deny/ask short circuit이 있다.
CONDITIONAL	namespace normalization과 consumer가 일치할 때 정책이 원하는 tool에 적용된다.
UNKNOWN	외부 server naming의 모든 collision은 확인되지 않는다.
NEXT READ	alias 추가 시 A32 normalization과 permission namespace를 함께 검증해야 한다.

다시 적용할 질문

1. 서로 다른 MCP server가 같은 normalized prefix를 만들면 coarse rule은 누구에게 적용되는가?
2. coarse allow가 command-level deny를 건너뛰게 해서는 안 되는 이유는 무엇인가?

SOURCE READING · 1/2

OBSERVED

Shell command는 문자열이 아니라 외부 언어다

quote, redirection, pipeline, substitution 때문에 prefix 비교만으로 effect를 판단할 수 없다.

bash permission entry 일부

src/tools/BashTool/bashPermissions.ts:1185-1225

```
1185    toolPermissionContext: ToolPermissionContext,
1186    commandPrefixResult: CommandPrefixResult | null | undefined,
1187    compoundCommandHasCd?: boolean,
1188    astParseSucceeded?: boolean,
1189  ): Promise<PermissionResult> {
1190    // 1. Check exact match first
1191    const exactMatchResult = bashToolCheckExactMatchPermission(
1192      input,
1193      toolPermissionContext,
1194    )
1195    if (exactMatchResult.behavior !== 'passthrough') {
1196      return exactMatchResult
1197    }
1198
1199    // 2. Check the command prefix
1200    const permissionResult = bashToolCheckPermission(
1201      input,
1202      toolPermissionContext,
1203      compoundCommandHasCd,
1204    )
1205    // 2a. Deny/ask if command was explictly denied/asked
1206    if (
1207      permissionResult.behavior === 'deny' ||
1208      permissionResult.behavior === 'ask'
1209    ) {
… 11 lines omitted; use the source link for the complete range …
1221      const safetyResult = await bashCommandIsSafeAsync(input.command)
1222
1223      if (safetyResult.behavior !== 'passthrough') {
1224        const decisionReason: PermissionDecisionReason = {
1225          type: 'other' as const,
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	parser 결과
그 표현이 만든 조건	subcommand별 PermissionResult
그 표현이 만든 조건	path/redirection 검사
그 표현이 만든 조건	sandbox override

언어와 런타임은 여기까지 보장한다

shell은 자체 grammar와 expansion semantics를 가진다. host 언어의 string API로 전부 재현하기 어렵다.

프로젝트가 이 메커니즘에 맡긴 책임

BashTool이 거대한 이유는 하나의 tool이 OS language adapter이기 때문이다.

경계를 잘못 읽을 때 생기는 위험

unknown syntax에서 fail-open하면 parser coverage 밖의 effect가 실행된다.

이 설명을 확인할 코드 src/tools/BashTool/bashPermissions.ts:1185-1225

ALGORITHM A21 · 독자의 질문

Compound command decision 합성은 어떤 시스템 문제를 닫는가

shell 문자열 하나에는 pipeline, redirection, compound command가 섞이고 일부 subcommand만 허용될 수 있다. 전체 문자열 matching은 실제 effect 단위를 놓친다.

코드가 필요해지는 이유

pipeline/compound command 일부만 허용되거나 거부될 때 전체 command authority를 정한다.

OBSERVED — bashPermissions가 parsed subcommand와 각 PermissionResult를 map으로 모아 전체 decision을 합성한다.

INFERENCE — shell parser가 의미 단위를 만들고 permission combiner가 그 단위들의 authority를 닫는다.

이 절에서 답할 결정 질문

subcommand별 decision을 합칠 때 deny·ask·allow 중 어떤 결론이 전체 command authority가 되는가?

이 설명을 확인할 코드 src/tools/BashTool/bashPermissions.ts:2430-2485

ALGORITHM A21 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/tools/BashTool/bashPermissions.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01
deny-overrides와 구성적 분석

부분 중 하나라도 금지라면 전체 pipeline이 그 effect를 포함하므로 deny가 전체를 지배한다. 단, parser가 실제 shell expansion을 같은 단위로 해석한다는 전제가 필요하다.

> **실제 코드로 돌아가면.** 2430–2485행에서 각 result의 precedence와 map key가 실제 실행 plan을 충분히 구분하는지 본다.

이론은 정답이 아니라 코드에 던질 질문이다

권한·신뢰 경계
`meet/deny-overrides policy · language sandboxing`
누가 어떤 근거로 effect를 허용하며, 더 약한 권한으로만 위임된다는 사실을 어디서 확인하는가?

상태·계산 모델
`compositional analysis`
단계의 순서나 표현을 바꿔도 같은 상태와 불변조건에 도달하는가?

ALGORITHM A21 · 실제 코드

OBSERVED

Compound command decision 합성의 결정점은 어디에 있는가

복합 shell 명령은 subcommand별 결과를 계산하고 전체 decision으로 접는다.

입력 표현. parsed subcommands, 그리고 각 PermissionResult

보존되는 작업 상태. Map<command, result>, 그리고 전체 deny/ask/allow 후보

subcommand result aggregation

src/tools/BashTool/bashPermissions.ts:2430-2485

```
2430      }
2431      return result
2432    }
2433
2434    // Check subcommand permission results
2435    const subcommandResults: Map<string, PermissionResult> = new Map()
2436    for (const subcommand of subcommands) {
2437      subcommandResults.set(
2438        subcommand,
2439        await checkCommandAndSuggestRules(
2440          {
2441            // Pass through input params like `sandbox`
2442            ...input,
2443            command: subcommand,
… 39 lines omitted; use the source link for the complete range …
2483            : undefined
2484
2485        const rules = extractRules(updates)
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A21 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

parsed subcommands, 그리고 각 PermissionResult은 처음부터 신뢰된 실행 상태가 아니다. Map<command, result>, 그리고 전체 deny/ask/allow 후보가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** 각 subcommand를 독립 검사한다. 이 결과가 다음 판단 "deny가 있으면 전체 실행을 막는다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** deny가 있으면 전체 실행을 막는다. 이 결과가 다음 판단 "ask/passthrough 항목을 사용자 preview에 모은다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** ask/passthrough 항목을 사용자 preview에 모은다. 이 결과가 다음 판단 "모든 항목이 허용될 때만 전체 allow를 만든다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** 모든 항목이 허용될 때만 전체 allow를 만든다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

부분 allow로 compound command를 실행하지 않는다. 사용자에게 승인 필요한 부분을 숨기지 않는다. parser가 분해하지 못하면 보수적으로 처리한다.

ALGORITHM A21 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **권한·신뢰 경계의 질문으로 읽기.** 부분 allow로 compound command를 실행하지 않는다.
- **상태·계산 모델의 질문으로 읽기.** 사용자에게 승인 필요한 부분을 숨기지 않는다.
- **권한·신뢰 경계의 질문으로 읽기.** parser가 분해하지 못하면 보수적으로 처리한다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- parser와 실제 shell expansion 차이
- redirection target 누락
- environment variable로 executable 변경

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 명령 전체 문자열보다 semantic component를 기준으로 authority를 계산한다.

RECOMMENDATION 실제 shell AST와 execution plan 차이를 negative corpus로 지속 검증한다.

ALGORITHM A21 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

command를 semantic component로 나누고 각 rule result를 계산한 뒤 전체 permission을 합친다.

effect가 시작되고 관찰되는 경계

승인된 전체 문자열이 shell에 넘어가므로 parser와 runtime shell 차이가 authority gap이 된다.

현재 설계가 선택한 trade-off와 비용

명령 전체 문자열보다 semantic component를 기준으로 authority를 계산한다. 이 선택의 계산·운영 비용은 subcommand 수 S에 선형 + 각 matcher 비용.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

expansion, redirection target, command substitution을 parser가 다르게 보면 허용하지 않은 effect가 생긴다. 이때 함께 대입할 반례는 parser와 실제 shell expansion 차이, redirection target 누락, 그리고 environment variable로 executable 변경이다. 이들을 모두 하나의 “실패”로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

원문, parsed AST/component, component decision, final decision을 함께 보존해야 한다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A21 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	subcommand result map과 deny/ask/allow 합성 경로가 존재한다.
CONDITIONAL	parser가 실제 shell effect boundary를 보수적으로 포착할 때 전체 authority가 안전하다.
UNKNOWN	모든 shell dialect와 expansion negative case의 완전성은 알 수 없다.
NEXT READ	새 shell syntax 지원 시 negative corpus와 실제 execution plan 비교를 먼저 확장해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. 허용된 command의 redirection target만 금지 경로라면 어느 component가 deny를 만들어야 하는가?
2. 두 subcommand 중 하나가 ask이고 하나가 allow이면 사용자에게 어떤 전체 payload를 보여 줘야 하는가?

이 설명을 확인할 코드 src/tools/BashTool/bashPermissions.ts:2430-2485

DISTINCTION TABLE

OBSERVED

Permission과 sandbox

두 control은 서로 다른 질문에 답한다.

층	보호 대상	남는 위험
Permission	subject authority	승인된 명령의 과도한 effect
Validation	input/target correctness	외부 race
Sandbox	filesystem/network/process 범위	sandbox escape/구성 오류
Audit	사후 설명과 복구	effect 자체 예방 불가

이 표를 읽는 관점

방어층을 여러 개 둔 이유는 실패 모델이 서로 다르기 때문이다.

CONCEPT IN CONTEXT

INFERENCE

Path는 문자열이 아니라 시간에 따라 변하는 resource identity다

normalize와 root check가 끝난 뒤 symlink나 외부 process가 target을 바꿀 수 있다.

코드에서 읽어낼 수 있는 단서

출발 단서	lexical path와 realpath가 다를 수 있다.
연결되는 단서	check와 use 사이에 await가 있으면 같은 event loop의 다른 task도 개입할 수 있다.
연결되는 단서	동기 구간이어도 다른 process race는 남는다.

이 단서들을 하나의 설계로 묶으면

TOCTOU를 완전히 막으려면 file descriptor, no-follow, atomic rename, version token 같은 OS primitive가 필요하다.

시스템 이해에 남는 결론

보고서에서는 "검사했다"가 아니라 검사한 identity와 실제 사용한 identity의 연결을 설명해야 한다.

이 설명을 확인할 코드 src/utils/permissions/filesystem.ts:1-80

04 Hook와 sandbox는 확장된 effect 경계를 만든다

사용자·plugin·managed policy가 삽입한 외부 프로그램과 OS 격리를 단순 옵션으로 보지 않고, tool input과 authority가 바뀌는 별도 protocol로 읽는다.

이 장의 논증은 다음 질문을 차례로 해결한다

코드	책임 연결 Hook event 이름은 단순 notification이 아니라 서로 다른 제어권을 뜻한다	알고리즘	판단 경계 Hook fan-out과 permission decision 합성
코드	판단 경계 Hook output은 JSON parse 뒤에도 event별 재검증을 거친다	코드	책임 연결 HTTP hook는 URL뿐 아니라 DNS·proxy·header secret 경계를 함께 검증한다
알고리즘	진입 경계 설정과 policy를 sandbox runtime capability로 투영	코드	책임 연결 excludedCommands는 security boundary가 아니라 sandbox 선택 heuristic이다
구분	효과·위임 외부 제어가 effect를 바꾸는 네 경계	사례	효과·위임 Bash tool 하나가 hook와 sandbox를 모두 지나는 경우

코드 독해의 시작점

`src/utils/hooks.ts` → `src/utils/hooks/**` → `src/utils/sandbox/sandbox-adapter.ts` → `src/tools/BashTool/shouldUseSandbox.ts`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

SOURCE READING · 1/2

OBSERVED

Hook event 이름은 단순 notification 이 아니라 서로 다른 제어권을 뜻한다

PreToolUse, PostToolUse, PermissionRequest, Stop, Compact, Task event는 같은 callback signature가 아니라 각기 다른 입력·차단·context 주입·retry 의미를 가진다.

tool 관련 hook contract metadata

src/utils/hooks/hooksConfigManager.ts:26-70

```
26    export const getHookEventMetadata = memoize(
27      function (toolNames: string[]): Record<HookEvent, HookEventMetadata> {
28        return {
29          PreToolUse: {
30            summary: 'Before tool execution',
31            description:
32              'Input to command is JSON of tool call arguments.\nExit code 0 - stdout/stderr not shown\nExit code 2 - show stderr to model and block tool call\nOther exit codes - show stderr to user only but continue with tool call',
33            matcherMetadata: {
34              fieldToMatch: 'tool_name',
35              values: toolNames,
36            },
37          },
38          PostToolUse: {
39            summary: 'After tool execution',
40            description:
41              'Input to command is JSON with fields "inputs" (tool call arguments) and "response" (tool call response).\nExit code 0 - stdout shown in transcript mode (ctrl+o)\nExit code 2 - show stderr to model immediately\nOther exit codes - show stderr to user only',
42            matcherMetadata: {
43              fieldToMatch: 'tool_name',
44              values: toolNames,
45            },
46          },
47          PostToolUseFailure: {
48            summary: 'After tool execution fails',
49            description:

… 16 lines omitted; use the source link for the complete range …

66            summary: 'When notifications are sent',
67            description:
68              'Input to command is JSON with notification message and type.\nExit code 0 - stdout/stderr not shown\nOther exit codes - show stderr to user only',
69            matcherMetadata: {
70              fieldToMatch: 'notification_type',
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	PreToolUse의 exit 2는 tool call을 block한다.
그 표현이 만든 조건	PostToolUse는 이미 발생한 effect 뒤에 실행되므로 rollback 권한이 없다.
그 표현이 만든 조건	PermissionDenied는 retry 요청을 model loop에 되돌릴 수 있다.
그 표현이 만든 조건	event별 matcher field와 허용 output이 다르다.

언어와 런타임은 여기까지 보장한다

event name과 hookSpecificOutput의 literal tag는 runtime protocol의 discriminant다. 동일한 JSON field라도 어느 lifecycle point에서 해석되는지에 따라 effect 전·후의 의미가 달라지며, exit status와 structured JSON은 두 개의 completion channel로 합성된다.

프로젝트가 이 메커니즘에 맡긴 책임

프로젝트는 hook을 임의 callback이 아니라 turn·tool·permission·task lifecycle에 끼어드는 extension protocol로 만든다. 따라서 event 추가는 config UI만이 아니라 schema, executor, projection, trust gate, failure disposition을 함께 바꾼다.

경계를 잘못 읽을 때 생기는 위험

PostToolUse의 block을 effect rollback처럼 설명하거나, PreToolUse의 updatedInput을 permission 검토 뒤에 적용하면 사용자가 승인한 input과 실행 input이 달라질 수 있다.

이 설명을 확인할 코드 src/utils/hooks/hooksConfigManager.ts:26-70

ALGORITHM A42 · 독자의 질문

Hook fan-out과 permission decision 합성은 어떤 시스템 문제를 닫는가

여러 hook가 병렬로 실행돼 input과 permission을 바꿀 수 있다. latency는 줄지만 두 hook의 updatedInput이 충돌하면 completion 순서가 우연히 최종 authority를 결정할 수 있다.

코드가 필요해지는 이유

서로 다른 source와 실행 방식(command, prompt, agent, HTTP, callback)의 hook을 한 lifecycle event에서 실행하면서 core effect의 최종 input과 authority를 결정한다.

OBSERVED — hook runner가 matchingHooks, timeout/result, outcome counts, aggregated permissionBehavior와 updated input을 소유한다.

INFERENCE — 각 hook는 확장 effect를 수행하지만 aggregate layer가 core pipeline에 반영할 하나의 decision을 만든다.

이 절에서 답할 결정 질문

병렬 fan-out 결과를 어떤 보수적 precedence로 합치고 input mutation의 provenance를 어떻게 보존하는가?

이 설명을 확인할 코드 src/utils/hooks.ts:1952-2075 src/utils/hooks.ts:2445-2685 src/utils/hooks.ts:2760-2895

ALGORITHM A42 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/utils/hooks.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01
fan-out/fan-in과 authority lattice

작업은 병렬로 퍼질 수 있지만 authority는 단순 last-writer-wins가 아니라 deny가 더 강한 보수적 순서로 합쳐야 한다. 데이터 mutation은 authority와 별도로 conflict를 검출해야 한다.

> **실제 코드로 돌아가면.** hook 결과 수집과 permissionBehavior 합성, updatedInput 선택이 같은 규칙을 쓰는지 구분한다.

이론은 정답이 아니라 코드에 던질 질문이다

검증·관측·운영
`fan-out`
최종 출력뿐 아니라 실패·중단·복구 경로의 보장을 어떤 관찰과 테스트로 입증할 수 있는가?

프로토콜·식별
`tagged union`
생산자와 소비자가 동일한 값·작업·종료 의미를 해석한다는 증거가 있는가?

권한·신뢰 경계
`deny`
누가 어떤 근거로 effect를 허용하며, 더 약한 권한으로만 위임된다는 사실을 어디서 확인하는가?

ALGORITHM A42 · 최소 예제

두 hook의 병렬 완료 순서는 input authority가 될 수 없다

서로 다른 path를 제안하는 hook 결과를 축소한다.

설명을 위한 최소 실행 예제

```
results = await Promise.all([
  hookA({ path: 'a' }), // allow, path='safe/a'
  hookB({ path: 'a' })  // deny,  path='blocked/a'
])
// deny는 precedence로 합치되 두 updatedInput은 conflict로 남겨야 함
```

관찰 시점	값과 상태	프로젝트 코드에서의 의미
fan-out	같은 base input을 두 hook가 본다.	각 결과는 독립 provenance를 가진다.
fan-in	deny가 allow보다 강하다.	completion order가 authority를 바꾸지 않는다.
mutation	두 updatedInput이 다르다.	하나를 우연히 선택하지 말고 conflict를 surface해야 한다.

예제를 실제 구현에 다시 연결한다

실제 aggregate가 behavior와 input을 서로 다른 규칙으로 다루는지 확인한다.

ALGORITHM A42 · 실제 코드

OBSERVED

Hook fan-out과 permission decision 합성의 결정점은 어디에 있는가

일치하는 여러 hook을 병렬 실행하되 각 결과를 progress·context·input mutation·permission lattice로 순차 투영한다.

입력 표현. typed HookInput, matcher query, ToolUseContext, AbortSignal, 그리고 source별 hook config

보존되는 작업 상태. matchingHooks, hook별 timeout/result, outcome counts, aggregated permissionBehavior, 그리고 additional contexts와 updated input

trust gate와 matching hook 준비

src/utils/hooks.ts:1952-2075

```
1952  async function* executeHooks({
1953    hookInput,
1954    toolUseID,
1955    matchQuery,
1956    signal,
1957    timeoutMs = TOOL_HOOK_EXECUTION_TIMEOUT_MS,
1958    toolUseContext,
1959    messages,
1960    forceSyncExecution,
1961    requestPrompt,
1962    toolInputSummary,
1963  }: {
1964    hookInput: HookInput
1965    toolUseID: string
… 107 lines omitted; use the source link for the complete range …
2073
2074    // Log hook execution start to OTEL (only for beta tracing)
2075    if (isBetaTracingEnabled()) {
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A42 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

typed HookInput, matcher query, ToolUseContext, AbortSignal, 그리고 source별 hook config은 처음부터 신뢰된 실행 상태가 아니다. matchingHooks, hook별 timeout/result, outcome counts, aggregated permissionBehavior, 그리고 additional contexts와 updated input가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	진입 조건을 해석한다 disable flag와 workspace trust를 먼저 검사한다. 이 결과가 다음 판단 "event·matcher·session에 맞는 hook을 찾아 중복을 제거한다."의 전제가 된다.
표현 정규화	표현과 선택지를 좁힌다 event·matcher·session에 맞는 hook을 찾아 중복을 제거한다. 이 결과가 다음 판단 "각 hook의 progress observation을 먼저 yield한다."의 전제가 된다.
유효성 판정	표현과 선택지를 좁힌다 각 hook의 progress observation을 먼저 yield한다. 이 결과가 다음 판단 "hook 종류별 adapter를 병렬 실행하고 JSON/exit status를 typed result로 바꾼다."의 전제가 된다.
결정 확정	표현과 선택지를 좁힌다 hook 종류별 adapter를 병렬 실행하고 JSON/exit status를 typed result로 바꾼다. 이 결과가 다음 판단 "결과 stream을 합치며 blocking error, context, input mutation을 투영한다."의 전제가 된다.
효과 조정	표현과 선택지를 좁힌다 결과 stream을 합치며 blocking error, context, input mutation을 투영한다. 이 결과가 다음 판단 "permission은 deny가 ask보다, ask가 allow보다 우선하도록 합성한다."의 전제가 된다.
결과 기록	표현과 선택지를 좁힌다 permission은 deny가 ask보다, ask가 allow보다 우선하도록 합성한다. 이 결과가 다음 판단 "duration과 outcome을 telemetry에 정산한다."의 전제가 된다.
상태 인계	후속 owner가 읽을 상태를 남긴다 duration과 outcome을 telemetry에 정산한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

신뢰되지 않은 workspace의 hook은 실행하지 않는다. 한 hook의 non-blocking failure가 다른 hook 결과를 지우지 않는다. deny 결과는 이후 allow 결과로 약화되지 않는다. updatedInput은 그것을 낸 hook의 decision 의미와 함께 전달된다.

ALGORITHM A42 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **검증·관측·운영의 질문으로 읽기.** 신뢰되지 않은 workspace의 hook은 실행하지 않는다.
- **권한·신뢰 경계의 질문으로 읽기.** 한 hook의 non-blocking failure가 다른 hook 결과를 지우지 않는다.
- **프로토콜·식별의 질문으로 읽기.** deny 결과는 이후 allow 결과로 약화되지 않는다.
- **검증·관측·운영의 질문으로 읽기.** updatedInput은 그것을 낸 hook의 decision 의미와 함께 전달된다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- 두 hook이 서로 다른 updatedInput을 내며 last-observed 결과가 우연히 승리
- async hook 결과가 effect 뒤늦게 도착
- trust check가 일부 executor에서 누락
- parallel hook의 side effect 충돌

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED extension latency를 줄이기 위해 실행은 병렬화하고 authority는 보수적 precedence로 합성한다.

RECOMMENDATION updatedInput에 source와 base-input digest를 붙이고 여러 mutation의 conflict를 검출하며, blocking/async 가능 여부를 event schema에 명시한다.

ALGORITHM A42 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

matching hook를 병렬 실행하고 timeout/result를 모아 permission과 context/input 변형을 합성한다.

effect가 시작되고 관찰되는 경계

hook 자체 side effect와 core tool effect가 다른 시간축에서 발생할 수 있다.

현재 설계가 선택한 trade-off와 비용

extension latency를 줄이기 위해 실행은 병렬화하고 authority는 보수적 precedence로 합성한다. 이 선택의 계산·운영 비용은 hook 수 H의 실행은 wall time상 병렬이지만 결과 처리 O(H); 외부 command·model·HTTP 비용이 지배한다.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

last-observed mutation, late async result, trust check 누락은 승인·실행 경계를 뒤집는다. 이때 함께 대입할 반례는 두 hook이 서로 다른 updatedInput을 내며 last-observed 결과가 우연히 승리, async hook 결과가 effect 뒤늦게 도착, trust check가 일부 executor에서 누락, 그리고 parallel hook의 side effect 충돌이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

hook id, base digest, result digest, blocking/async disposition과 aggregate reason을 남겨야 한다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A42 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	병렬 hook fan-out과 보수적 permission 합성이 구현돼 있다.
CONDITIONAL	executor가 trust/timeout 규칙을 동일하게 적용하고 mutation conflict가 의미 있게 처리될 때 안전하다.
UNKNOWN	외부 hook의 실제 side effect와 production conflict 빈도는 source로 확인할 수 없다.
NEXT READ	updatedInput provenance와 conflict detector를 추가하기 전 모든 executor의 aggregation 경로를 대조해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. deny hook가 timeout 뒤 도착하면 core effect는 이미 시작될 수 있는가?
2. 두 allow hook가 서로 다른 updatedInput을 만들면 어떤 값을 사용자에게 다시 승인받아야 하는가?

이 설명을 확인할 코드 src/utils/hooks.ts:1952-2075 src/utils/hooks.ts:2445-2685 src/utils/hooks.ts:2760-2895

SOURCE READING · 1/2

OBSERVED

Hook output은 JSON parse 뒤에도 event별 재검증을 거친다

approve/block, permissionDecision, updatedInput, additionalContext는 한 object에서 읽히지만 expected hook event와 허용된 variant를 확인한 뒤에만 core state로 들어간다.

hook JSON output의 event별 해석 src/utils/hooks.ts:520-670

```
520       if (syncJson.stopReason) {
521         result.stopReason = syncJson.stopReason
522       }
523     }
524
525     if (json.decision) {
526       switch (json.decision) {
527         case 'approve':
528           result.permissionBehavior = 'allow'
529           break
530         case 'block':
531           result.permissionBehavior = 'deny'
532           result.blockingError = {
533             blockingError: json.reason || 'Blocked by hook',
534             command,
535           }
536           break
537         default:
538           // Handle unknown decision types as errors
539           throw new Error(
540             `Unknown hook decision type: ${json.decision}. Valid types are: approve, block`,
541           )
542       }
543     }
544
… 121 lines omitted; use the source link for the complete range …
666             if (
667               json.hookSpecificOutput.decision.behavior === 'allow' &&
668               json.hookSpecificOutput.decision.updatedInput
669             ) {
670               result.updatedInput = json.hookSpecificOutput.decision.updatedInput
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	legacy decision과 hookSpecificOutput permissionDecision을 각각 해석한다.
그 표현이 만든 조건	expectedHookEvent와 실제 hookEventName이 다르면 오류로 거부한다.
그 표현이 만든 조건	PreToolUse만 updatedInput과 permission decision을 제공한다.
그 표현이 만든 조건	PermissionRequest와 Elicitation은 별도의 structured response를 만든다.

언어와 런타임은 여기까지 보장한다

Zod/타입 union은 허용된 shape를 제한하지만 event context에 맞는 semantic field인지는 추가 dispatch가 판정한다. parsing, validation, authorization은 같은 단계가 아니다.

프로젝트가 이 메커니즘에 맡긴 책임

프로젝트는 외부 process stdout을 곧바로 prompt나 tool input으로 쓰지 않고 typed HookResult로 lower한다. 이는 hook script와 core runtime 사이의 anti-corruption layer다.

경계를 잘못 읽을 때 생기는 위험

schema가 느슨해지거나 event tag check가 빠지면 PostToolUse output이 PreToolUse updatedInput처럼 해석되는 confused-deputy 문제가 생길 수 있다.

이 설명을 확인할 코드 src/utils/hooks.ts:520-670

SOURCE READING · 1/2

OBSERVED

HTTP hook는 URL뿐 아니라 DNS·proxy·header secret 경계를 함께 검증한다

외부 endpoint 호출은 allowlist, 환경 변수 최소 공개, header injection 방지, SSRF guard, sandbox proxy를 조합한다.

HTTP hook policy와 request 준비

src/utils/hooks/execHttpHook.ts:52-180

```
52    } {
53      const settings = settingsModule.getInitialSettings()
54      return {
55        allowedUrls: settings.allowedHttpHookUrls,
56        allowedEnvVars: settings.httpHookAllowedEnvVars,
57      }
58    }
59
60    /**
61     * Match a URL against a pattern with * as a wildcard (any characters).
62     * Same semantics as the MCP server allowlist patterns.
63     */
64    function urlMatchesPattern(url: string, pattern: string): boolean {
65      const escaped = pattern.replace(/[.+?^${}()|[\]\\]/g, '\\$&')
66      const regexStr = escaped.replace(/\*/g, '.*')
67      return new RegExp(`^${regexStr}$`).test(url)
68    }
69
70    /**
71     * Strip CR, LF, and NUL bytes from a header value to prevent HTTP header
72     * injection (CRLF injection) via env var values or hook-configured header
73     * templates. A malicious env var like "token\r\nX-Evil: 1" would otherwise
74     * inject a second header into the request.
75     */
76    function sanitizeHeaderValue(value: string): string {
… 99 lines omitted; use the source link for the complete range …
176       const sandboxProxy = await getSandboxProxyConfig()
177
178       // Detect env var proxy (HTTP_PROXY / HTTPS_PROXY, respecting NO_PROXY).
179       // When set, configureGlobalAgents() has already installed a request
180       // interceptor that sets httpsAgent to an HttpsProxyAgent — the proxy
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	allowedUrls가 undefined면 제한 없음, 빈 배열이면 모두 차단, 값이 있으면 pattern match를 요구한다.
그 표현이 만든 조건	hook allowlist와 managed allowedEnvVars의 교집합만 header interpolation에 사용한다.
그 표현이 만든 조건	CR·LF·NUL을 제거해 header injection을 막는다.
그 표현이 만든 조건	proxy가 없을 때 custom DNS lookup으로 private/link-local SSRF를 차단한다.

언어와 런타임은 여기까지 보장한다

문자열 URL 검사는 DNS resolution 뒤 실제 address의 신뢰성을 보장하지 않는다. 반대로 corporate/sandbox proxy가 DNS를 소유할 때 local lookup을 강제하면 정책을 잘못 적용하므로 transport owner에 따라 guard 위치를 바꾼다.

프로젝트가 이 메커니즘에 맡긴 책임

Hook는 arbitrary code extension이므로 HTTP adapter도 Tool과 같은 trust/authority 분석 대상이다. 환경 변수는 process에 존재한다는 이유만으로 hook에 공개되지 않는다.

경계를 잘못 읽을 때 생기는 위험

wildcard allowlist가 지나치게 넓거나 redirect 정책이 바뀌거나 proxy 경로에서 domain enforcement가 빠지면 secret exfiltration과 SSRF 방어가 갈라진다.

이 설명을 확인할 코드 src/utils/hooks/execHttpHook.ts:52-180

ALGORITHM A43 · 독자의 질문

설정과 policy를 sandbox runtime capability로 투영은 어떤 시스템 문제를 닫는가

prompt-level 허용만으로는 child process나 우회 경로의 파일·네트워크 접근을 막지 못한다. 여러 settings source의 규칙을 OS별 sandbox가 집행할 capability로 내려야 한다.

코드가 필요해지는 이유

여러 settings source의 filesystem/network 규칙을 OS별 sandbox runtime이 집행할 하나의 configuration으로 만든다.

OBSERVED — sandbox-adapter가 merged settings, policy source, cwd/worktree 정보를 effective runtime config로 만든다.

INFERENCE — permission UI는 의사결정을 소유하고 OS sandbox는 실제 집행을 소유하며 adapter가 두 표현을 연결한다.

이 절에서 답할 결정 질문

설정의 path/domain 규칙이 runtime capability로 투영될 때 넓어지거나 사라지는 권한은 무엇인가?

이 설명을 확인할 코드 src/utils/sandbox/sandbox-adapter.ts:172-260 src/utils/sandbox/sandbox-adapter.ts:704-799 src/tools/BashTool/shouldUseSandbox.ts:123-154

ALGORITHM A43 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/utils/sandbox/sandbox-adapter.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01
capability attenuation과 path identity

하위 runtime에는 필요한 권한보다 넓지 않은 capability만 내려야 한다. 문자열 prefix는 symlink·realpath·worktree 경계를 충분히 표현하지 못할 수 있다.

> **실제 코드로 돌아가면.** 172–260행에서 allow/deny path, domain, managed-only switch가 어떤 precedence로 runtime config가 되는지 본다.

이론은 정답이 아니라 코드에 던질 질문이다

권한·신뢰 경계
`capability attenuation · policy`
누가 어떤 근거로 effect를 허용하며, 더 약한 권한으로만 위임된다는 사실을 어디서 확인하는가?

구성·변형·성능
`configuration composition`
조립 방식이나 최적화를 바꿔도 핵심 계약이 adapter와 build 변형 안에서 보존되는가?

ALGORITHM A43 · 실제 코드

OBSERVED

설정과 policy를 sandbox runtime capability로 투영의 결정점은 어디에 있는가

permission rule과 sandbox setting의 서로 다른 path 문법을 해석하고 network·read·write capability를 runtime config로 낮춘다.

입력 표현. merged SettingsJson, source별 policy settings, 그리고 cwd·settings root·worktree 정보

보존되는 작업 상태. allowed/denied domains, allowWrite/denyWrite/allowRead/denyRead, settings-file deny list, 그리고 managed-only switches

sandbox runtime config projection

src/utils/sandbox/sandbox-adapter.ts:172-260

```
172  export function convertToSandboxRuntimeConfig(
173    settings: SettingsJson,
174  ): SandboxRuntimeConfig {
175    const permissions = settings.permissions || {}
176
177    // Extract network domains from WebFetch rules
178    const allowedDomains: string[] = []
179    const deniedDomains: string[] = []
180
181    // When allowManagedSandboxDomainsOnly is enabled, only use domains from policy settings
182    if (shouldAllowManagedSandboxDomainsOnly()) {
183      const policySettings = getSettingsForSource('policySettings')
184      for (const domain of policySettings?.sandbox?.network?.allowedDomains ||
185        []) {
… 72 lines omitted; use the source link for the complete range …
258    // HEAD + objects/ + refs/. An attacker planting these (plus a config with
259    // core.fsmonitor) escapes the sandbox when Claude's unsandboxed git runs.
260    //
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A43 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

merged SettingsJson, source별 policy settings, 그리고 cwd·settings root·worktree 정보은 처음부터 신뢰된 실행 상태가 아니다. allowed/denied domains, allowWrite/denyWrite/allowRead/denyRead, settings-file deny list, 그리고 managed-only switches가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** managed-only network/read policy가 authority를 독점하는지 결정한다. 이 결과가 다음 판단 "WebFetch permission rule과 sandbox domain list를 allowed/denied domain으로 변환한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** WebFetch permission rule과 sandbox domain list를 allowed/denied domain으로 변환한다. 이 결과가 다음 판단 "permission rule path와 sandbox filesystem path의 서로 다른 prefix 의미를 source 기준으로 resolve한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** permission rule path와 sandbox filesystem path의 서로 다른 prefix 의미를 source 기준으로 resolve한다. 이 결과가 다음 판단 "cwd와 temp를 write capability에 넣고 settings file은 항상 denyWrite에 넣는다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** cwd와 temp를 write capability에 넣고 settings file은 항상 denyWrite에 넣는다. 이 결과가 다음 판단 "worktree와 git metadata 등 runtime resource를 보강한다."의 전제가 된다.
효과 조정	**표현과 선택지를 좁힌다** worktree와 git metadata 등 runtime resource를 보강한다. 이 결과가 다음 판단 "BaseSandboxManager에 config를 전달하고 settings change에 맞춰 갱신한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** BaseSandboxManager에 config를 전달하고 settings change에 맞춰 갱신한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

managed-only가 켜지면 낮은 authority source가 allow 범위를 넓히지 못한다. settings file은 sandboxed command가 수정할 수 없다. permission `/path`와 sandbox filesystem `/path`의 다른 의미를 혼동하지 않는다. 새 config가 적용되기 전 pending command가 stale capability로 빠져나가지 않게 refresh 시점을 통제한다.

ALGORITHM A43 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **권한·신뢰 경계의 질문으로 읽기.** managed-only가 켜지면 낮은 authority source가 allow 범위를 넓히지 못한다.
- **구성·변형·성능의 질문으로 읽기.** settings file은 sandboxed command가 수정할 수 없다.
- **권한·신뢰 경계의 질문으로 읽기.** permission `/path`와 sandbox filesystem `/path`의 다른 의미를 혼동하지 않는다.
- **구성·변형·성능의 질문으로 읽기.** 새 config가 적용되기 전 pending command가 stale capability로 빠져나가지 않게 refresh 시점을 통제한다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- path prefix 오해로 의도보다 넓은 allow
- symlink/worktree real path drift
- initialization 실패를 graceful fallback으로 처리해 sandbox 없는 실행
- settings 갱신 race

이 반례가 나타나면 "알고리즘이 틀렸다"로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED prompt-level permission과 OS-level enforcement를 연결하되 별도 configuration semantics를 유지한다.

RECOMMENDATION resolved real path·source provenance·effective capability diff를 diagnostic으로 제공하고, initialization failure의 fail-open/fail-closed policy를 surface별로 명시한다.

ALGORITHM A43 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

settings를 읽어 filesystem/network capability와 settings-file deny list를 구성해 sandbox를 초기화한다.

effect가 시작되고 관찰되는 경계

초기화 성공 여부가 이후 모든 child effect의 실제 경계를 결정한다.

현재 설계가 선택한 trade-off와 비용

prompt-level permission과 OS-level enforcement를 연결하되 별도 configuration semantics를 유지한다. 이 선택의 계산·운영 비용은 rule과 path 수에 선형이며 glob normalization과 OS runtime setup 비용이 별도다.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

path prefix, symlink drift, fail-open fallback, settings race는 표시된 policy보다 넓은 실행을 허용할 수 있다. 이때 함께 대입할 반례는 path prefix 오해로 의도보다 넓은 allow, symlink/worktree real path drift, initialization 실패를 graceful fallback으로 처리해 sandbox 없는 실행, 그리고 settings 갱신 race이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

resolved realpath, source provenance, effective diff, initialization disposition을 남겨야 한다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A43 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	여러 설정에서 filesystem/network sandbox config를 투영하는 adapter가 있다.
CONDITIONAL	지원 OS와 초기화 성공, path resolution이 예상대로일 때 capability가 집행된다.
UNKNOWN	배포 surface별 fail-open/fail-closed 정책과 OS backend 동작은 이 snapshot에서 완결되지 않는다.
NEXT READ	fallback을 바꾸기 전 surface별 초기화 실패 경로와 실제 sandbox backend를 함께 검증해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. 허용 path 안의 symlink가 밖을 가리키면 문자열 prefix와 realpath 결과는 어떻게 달라지는가?
2. sandbox 초기화 실패 뒤 일반 permission prompt만으로 계속 실행해도 같은 보장인가?

이 설명을 확인할 코드 src/utils/sandbox/sandbox-adapter.ts:172-260 src/utils/sandbox/sandbox-adapter.ts:704-799 src/tools/BashTool/shouldUseSandbox.ts:123-154

SOURCE READING · 1/2

OBSERVED

excludedCommands는 security boundary가 아니라 sandbox 선택 heuristic이다

사용자 편의를 위한 제외 목록과 unsandboxed command를 허용하는 policy를 분리하지 않으면 설정 하나가 권한 상승처럼 보이거나 실제 통제를 놓친다.

excluded command의 비권위적 분류 src/tools/BashTool/shouldUseSandbox.ts:1-82

```
import { getFeatureValue_CACHED_MAY_BE_STALE } from 'src/services/analytics/growthbook.js'
import { splitCommand_DEPRECATED } from '../../utils/bash/commands.js'
import { SandboxManager } from '../../utils/sandbox/sandbox-adapter.js'
import { getSettings_DEPRECATED } from '../../utils/settings/settings.js'
import {
  BINARY_HIJACK_VARS,
  bashPermissionRule,
  matchWildcardPattern,
  stripAllLeadingEnvVars,
  stripSafeWrappers,
} from './bashPermissions.js'

type SandboxInput = {
  command?: string
  dangerouslyDisableSandbox?: boolean
}

// NOTE: excludedCommands is a user-facing convenience feature, not a security boundary.
// It is not a security bug to be able to bypass excludedCommands — the sandbox permission
// system (which prompts users) is the actual security control.
function containsExcludedCommand(command: string): boolean {
  // Check dynamic config for disabled commands and substrings (only for ants)
  if (process.env.USER_TYPE === 'ant') {
    const disabledCommands = getFeatureValue_CACHED_MAY_BE_STALE<{
      commands: string[]
```

… 52 lines omitted; use the source link for the complete range …

```
        // We iteratively apply both stripping operations until no new candidates are
        // produced (fixed-point), matching the approach in filterRulesByContentsMatchingInput.
        // This handles interleaved patterns like `timeout 300 FOO=bar bazel run`
        // where single-pass composition would fail.
        const candidates = [trimmed]
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	파일 주석이 excludedCommands는 보안 경계가 아니라고 명시한다.
그 표현이 만든 조건	compound command를 분해해 각 subcommand를 검사한다.
그 표현이 만든 조건	env assignment와 safe wrapper를 fixed point로 벗겨 후보를 만든다.
그 표현이 만든 조건	최종 shouldUseSandbox는 명시 override와 policy 허용을 함께 본다.

언어와 런타임은 여기까지 보장한다

command normalization heuristic은 shell language의 완전한 parser나 OS enforcement가 아니다. 여러 문자열 후보를 만들면 흔한 wrapper 회피는 줄지만 alias, expansion, subshell의 전체 의미를 증명하지 못한다.

프로젝트가 이 메커니즘에 맡긴 책임

프로젝트는 “sandbox에 넣을지”의 routing과 “sandbox 밖 실행이 허용되는지”의 authority를 별도 질문으로 둔다. 보고서는 heuristic hit를 보안 허가로 표현하면 안 된다.

경계를 잘못 읽을 때 생기는 위험

새 wrapper나 compound syntax가 후보 생성에서 빠지면 편의 목록이 예상과 다르게 동작한다. 더 위험한 문제는 호출자가 이 false를 별도 permission 없이 곧바로 실행 허가로 오해하는 경우다.

이 설명을 확인할 코드 src/tools/BashTool/shouldUseSandbox.ts:1-82

DISTINCTION TABLE

OBSERVED

외부 제어가 effect를 바꾸는 네 경계

Hook와 sandbox를 하나의 "보안 기능"으로 합치지 않고 각자의 개입 시점과 권한을 구분한다.

경계	바꾸는 것	바꿀 수 없는 것
PreToolUse hook	input·allow/ask/deny·context	아직 발생하지 않은 effect의 결과
PermissionRequest hook	사용자 prompt에 대한 allow/deny 응답	OS sandbox policy 자체
Sandbox config	filesystem/network capability	model의 tool protocol identity
PostToolUse hook	관찰·context·일부 MCP output projection	이미 commit된 외부 effect

이 표를 읽는 관점

한 경계의 성공을 다른 경계의 성공으로 대체할 수 없다. permission allow는 sandbox capability를 자동 확대하지 않고, sandbox success는 hook output의 신뢰성을 증명하지 않는다.

SELECTIVE EXECUTION TRACE

OBSERVED

Bash tool 하나가 hook와 sandbox를 모두 지나는 경우

명령 문자열이 실제 process effect가 되기 전후의 authority를 따라간다.

효과·위임

PreToolUse

matching hook가 input을 수정하거나 deny/ask/allow를 제안한다.

이때 남은 상태와 판단이 "Permission"에서 사용할 전제가 된다.

판단 경계

Permission

수정된 최종 input에 rule·user decision을 적용한다.

이때 남은 상태와 판단이 "Sandbox routing"에서 사용할 전제가 된다.

책임 연결

Sandbox routing

override policy와 excluded-command heuristic을 평가한다.

이때 남은 상태와 판단이 "Sandbox runtime"에서 사용할 전제가 된다.

진입 경계

Sandbox runtime

resolved filesystem/network capability로 command를 감싼다.

이때 남은 상태와 판단이 "Process effect"에서 사용할 전제가 된다.

진입 경계

Process effect

shell process를 실행하고 output·exit를 수집한다.

이때 남은 상태와 판단이 "PostToolUse"에서 사용할 전제가 된다.

효과·위임

PostToolUse

이미 발생한 결과를 관찰하고 추가 context를 만든다.

이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

각 단계는 동일한 최종 command identity와 source provenance를 공유해야 하며, 앞 단계가 바꾼 input은 뒤 단계에서 다시 검증되어야 한다.

어디에서 설명이 틀어지는가

승인 이후 hook가 input을 바꾸거나 sandbox config refresh 전에 command가 시작되면 preview, authority, effect가 서로 다른 명령을 가리킨다.

이 설명을 확인할 코드 src/utils/hooks.ts:3390-3485 src/utils/sandbox/sandbox-adapter.ts:704-799

05 파일 변경을 transaction처럼 다룬다

원자적 database transaction은 아니지만 관찰·검증·commit·사후 projection을 분리한다.

이 장의 논증은 다음 질문을 차례로 해결한다

알고리즘	책임 연결 Read가 edit precondition을 만든다	알고리즘	책임 연결 FileEdit optimistic validation과 commit window
코드	판단 경계 checkPermissions와 call은 서로 다른 시점의 world를 본다	사례	책임 연결 사례: config.ts edit의 결정 장부
구분	효과·위임 Effect 결과의 네 상태	해석	효과·위임 권장 분리: Effect·Policy·Presentation
알고리즘	효과·위임 Tool contract characterization		

코드 독해의 시작점

`src/tools/FileReadTool` → `src/tools/FileEditTool` → `src/utils/fileStateCache.ts`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

ALGORITHM A22 · 독자의 질문

Read가 edit precondition을 만든다은 어떤 시스템 문제를 닫는가

모델이 파일을 읽은 뒤 편집하기 전 다른 process가 파일을 바꿀 수 있다. lock 없이 안전하게 쓰려면 읽기 자체가 나중 검증할 version observation을 남겨야 한다.

코드가 필요해지는 이유

모델이 읽은 버전과 실제 편집 대상 버전이 같은지 나중에 판단할 근거를 만든다.

OBSERVED FileReadTool이 content/mtime/range observation을 readFileState에 기록한다.

INFERENCE Read는 표시용 결과뿐 아니라 후속 Edit가 stale 여부를 판단하는 read set을 만든다.

이 절에서 답할 결정 질문

partial Read가 어떤 version token과 범위를 기록해야 Edit의 precondition이 되는가?

> ### 이 코드를 읽기 전에 잡을 개념
>
> **낙관적 동시성과 read set.** 긴 lock 대신 읽은 version을 기억하고 commit 직전에 현재 version과 비교한다. 일부 range만 읽었다면 전체 파일에 대한 전제를 자동으로 얻지 않는다.
>
> 1020–1040행에서 offset/limit read가 어떤 content와 mtime을 state에 남기는지 본다.

이 설명을 확인할 코드 src/tools/FileReadTool/FileReadTool.ts:1020-1040

ALGORITHM A22 · 실제 코드

OBSERVED

Read가 edit precondition을 만든다의 결정점은 어디에 있는가

FileRead는 출력만 반환하지 않고 이후 Edit가 검증할 file observation state를 남긴다.

입력 표현. path, offset, limit, readFileState, 그리고 token/file limits

보존되는 작업 상태. content/mtime/range observation, 그리고 tool result

range read와 readFileState observation 기록

src/tools/FileReadTool/FileReadTool.ts:1020-1040

```
1020    const lineOffset = offset === 0 ? 0 : offset - 1
1021    const { content, lineCount, totalLines, totalBytes, readBytes, mtimeMs } =
1022      await readFileInRange(
1023        resolvedFilePath,
1024        lineOffset,
1025        limit,
1026        limit === undefined ? maxSizeBytes : undefined,
1027        context.abortController.signal,
1028      )
1029
1030    await validateContentTokens(content, ext, maxTokens)
1031
1032    readFileState.set(fullFilePath, {
1033      content,
… 4 lines omitted; use the source link for the complete range …
1038    context.nestedMemoryAttachmentTriggers?.add(fullFilePath)
1039
1040    // Snapshot before iterating — a listener that unsubscribes mid-callback
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A22 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

path, offset, limit, readFileState, 그리고 token/file limits은 처음부터 신뢰된 실행 상태가 아니다. content/mtime/range observation, 그리고 tool result가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** path와 file type을 검사한다. 이 결과가 다음 판단 "내용 또는 image를 읽는다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** 내용 또는 image를 읽는다. 이 결과가 다음 판단 "token/size 제한을 적용한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** token/size 제한을 적용한다. 이 결과가 다음 판단 "관찰 state를 cache에 기록한다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** 관찰 state를 cache에 기록한다. 이 결과가 다음 판단 "model/UI용 result를 만든다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** model/UI용 result를 만든다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

partial read 범위를 구분한다. 관찰 시점과 path identity가 함께 남는다. 출력 truncation과 stored precondition을 혼동하지 않는다.

ALGORITHM A22 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **동시성·시간·합의의 질문으로 읽기.** partial read 범위를 구분한다.
- **권한·신뢰 경계의 질문으로 읽기.** 관찰 시점과 path identity가 함께 남는다.
- **동시성·시간·합의의 질문으로 읽기.** 출력 truncation과 stored precondition을 혼동하지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- mtime granularity
- partial read로 전체 file 전제
- symlink target 변경

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 별도 lock 없이 Read→Edit optimistic workflow를 지원한다.

RECOMMENDATION content hash 또는 inode/version token을 선택적으로 함께 기록한다.

ALGORITHM A22 · 운영과 현재 상태

작은 변환도 effect와 미확정 경계를 남긴다

파일 range를 읽고 결과와 version observation을 동시에 저장한다.

실패 뒤 남는 상태

mtime granularity, symlink target 변화, partial read의 과도한 일반화는 stale check를 약하게 한다.

OBSERVED	Read 결과가 readFileState observation을 남긴다.
CONDITIONAL	Edit가 같은 path identity와 충분한 version token을 비교할 때 stale overwrite를 줄인다.
UNKNOWN	filesystem별 mtime 해상도와 symlink race는 확인되지 않는다.
NEXT READ	stronger token 도입 전 FileEdit의 비교와 path normalization을 함께 본다.

다시 적용할 질문

1. 파일 앞부분만 읽고 뒤를 replace하면 read set이 충분한가?
2. 같은 경로 문자열의 inode가 바뀌면 mtime만으로 새 파일을 구분할 수 있는가?

ALGORITHM A23 · 독자의 질문

FileEdit optimistic validation과 commit window은 어떤 시스템 문제를 닫는가

Edit는 모델이 본 old_string을 현재 파일에 적용하지만 permission·history 같은 await 사이에 외부 변경이 끼어들 수 있다. commit window를 줄여도 완전한 lock은 아니다.

코드가 필요해지는 이유

모델이 읽은 old string을 현재 파일에 안전하게 적용하고 외부 변경을 덮어쓰지 않는다.

OBSERVED — FileEditTool이 current content, patched content, history snapshot과 write 순서를 소유한다.

INFERENCE — readFileState가 precondition을 제공하고 FileEdit가 검증·patch·commit을 한 critical section처럼 배열한다.

이 절에서 답할 결정 질문

stale validation과 동기 write 사이의 좁은 window에서 무엇이 보장되고 어떤 race가 남는가?

이 설명을 확인할 코드 src/tools/FileEditTool/FileEditTool.ts:427-491 src/tools/FileEditTool/FileEditTool.ts:599-625

ALGORITHM A23 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/tools/FileEditTool/FileEditTool.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01
compare-and-swap와 commit window

읽은 version이 아직 current일 때만 새 값을 쓴다는 것이 CAS의 직관이다. 검사와 write가 하나의 filesystem atomic primitive가 아니면 그 사이 race는 남는다.

> **실제 코드로 돌아가면.** 427–491행에서 await를 앞당긴 뒤 stale check부터 sync write까지 await가 없는 이유를 본다.

이론은 정답이 아니라 코드에 던질 질문이다

동시성·시간·합의
`optimistic concurrency control · critical section · compare-and-swap`

stale read, 중복 실행, 취소가 끼어들어도 effect와 최종 상태의 의미가 보존되는가?

ALGORITHM A23 · 최소 예제

검사 직후 외부 write가 끼면 optimistic check만으로는 덮어쓰기를 막지 못한다

두 writer가 같은 v1을 읽는 interleaving을 본다.

설명을 위한 최소 실행 예제

```
A reads v1
B reads v1
A validates v1
B writes v2
A writes v3 // B의 변경을 덮음
```

관찰 시점	값과 상태	프로젝트 코드에서의 의미
공통 read	A와 B 모두 v1 관찰	둘 다 아직 valid하다고 믿는다.
A validate	현재 v1 확인	atomic commit은 아직 아님
B→A write	v2 뒤 v3	validation과 write 사이 race가 드러난다.

예제를 실제 구현에 다시 연결한다

실제 구현이 await를 제거해 window를 줄이지만 외부 process를 lock하지는 않는다는 경계를 설명한다.

ALGORITHM A23 · 실제 코드

OBSERVED

FileEdit optimistic validation과 commit window의 결정점은 어디에 있는가

stale check 뒤 write까지 JS event loop에 제어를 넘기지 않는 좁은 동기 구간을 만든다.

입력 표현. file_path, old_string, new_string, replace_all, 그리고 read state

보존되는 작업 상태. current content, patched content, 그리고 history snapshot

await를 앞당긴 뒤 stale check→patch→동기 write

src/tools/FileEditTool/FileEditTool.ts:427-491

```
427       // Ensure parent directory exists before the atomic read-modify-write section.
428       // These awaits must stay OUTSIDE the critical section below — a yield between
429       // the staleness check and writeTextContent lets concurrent edits interleave.
430       await fs.mkdir(dirname(absoluteFilePath))
431       if (fileHistoryEnabled()) {
432         // Backup captures pre-edit content — safe to call before the staleness
433         // check (idempotent v1 backup keyed on content hash; if staleness fails
434         // later we just have an unused backup, not corrupt state).
435         await fileHistoryTrackEdit(
436           updateFileHistoryState,
437           absoluteFilePath,
438           parentMessage.uuid,
439         )
440       }
… 48 lines omitted; use the source link for the complete range …
489
490       // 5. Write to disk
491       writeTextContent(absoluteFilePath, updatedFile, encoding, endings)
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A23 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

file_path, old_string, new_string, replace_all, 그리고 read state은 처음부터 신뢰된 실행 상태가 아니다. current content, patched content, 그리고 history snapshot가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** directory/history 준비 같은 async 작업을 먼저 끝낸다. 이 결과가 다음 판단 "commit 직전 현재 파일을 다시 읽는다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** commit 직전 현재 파일을 다시 읽는다. 이 결과가 다음 판단 "read state와 stale 여부를 검사한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** read state와 stale 여부를 검사한다. 이 결과가 다음 판단 "old string match와 uniqueness를 검증한다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** old string match와 uniqueness를 검증한다. 이 결과가 다음 판단 "patch를 계산하고 write한다."의 전제가 된다.
효과 조정	**표현과 선택지를 좁힌다** patch를 계산하고 write한다. 이 결과가 다음 판단 "LSP/history/UI observer를 갱신한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** LSP/history/UI observer를 갱신한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

check와 synchronous write 사이 내부 await가 없다. old_string mismatch는 write하지 않는다. write 뒤 observer가 이전 content를 canonical로 보지 않는다.

ALGORITHM A23 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **동시성·시간·합의의 질문으로 읽기.** check와 synchronous write 사이 내부 await가 없다.
- **동시성·시간·합의의 질문으로 읽기.** old_string mismatch는 write하지 않는다.
- **동시성·시간·합의의 질문으로 읽기.** write 뒤 observer가 이전 content를 canonical로 보지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- 외부 process race
- mtime/content token 약함
- observer update 실패
- replace_all 과잉 변경

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 긴 lock 대신 commit 직전 version validation을 쓰는 optimistic 전략이다.

RECOMMENDATION atomic temp-write/rename과 stronger version token을 검토하고 effect receipt에 before/after digest를 남긴다.

ALGORITHM A23 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

permission 등 비동기 작업을 먼저 끝내고 current version을 확인한 뒤 patch와 sync write를 연속 수행한다.

effect가 시작되고 관찰되는 경계

filesystem write가 commit이며 history/observer update는 후속 관찰 effect다.

현재 설계가 선택한 trade-off와 비용

긴 lock 대신 commit 직전 version validation을 쓰는 optimistic 전략이다. 이 선택의 계산·운영 비용은 파일 크기 N과 match 수에 따라 O(N).로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

외부 process race, 약한 token, observer failure, replace_all 과잉 변경은 successful return과 실제 보장을 갈라놓는다. 이때 함께 대입할 반례는 외부 process race, mtime/content token 약함, observer update 실패, 그리고 replace_all 과잉 변경이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

before/after digest, token, write receipt, observer disposition을 함께 남겨야 한다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A23 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	await를 앞당기고 stale check→patch→동기 write를 좁게 배열한다.
CONDITIONAL	같은 process event loop의 interleaving은 줄지만 외부 writer가 없다는 보장은 아니다.
UNKNOWN	filesystem atomicity와 외부 editor race 빈도는 source만으로 확정할 수 없다.
NEXT READ	temp-write/rename을 검토할 때 metadata와 symlink semantics, history update 순서도 함께 검증해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. write는 성공했지만 observer update가 실패하면 caller에게 어떤 receipt를 반환해야 하는가?
2. atomic rename이 content race를 줄여도 permission과 path identity는 왜 별도 문제인가?

이 설명을 확인할 코드 src/tools/FileEditTool/FileEditTool.ts:427-491 src/tools/FileEditTool/FileEditTool.ts:599-625

SOURCE READING · 1/2

OBSERVED

checkPermissions와 call은 서로 다른 시점의 world를 본다

permission preview 뒤 filesystem이 바뀔 수 있으므로 authorization과 effect validation을 다시 연결해야 한다.

FileEdit definition과 permission

src/tools/FileEditTool/FileEditTool.ts:86-160

```
86    export const FileEditTool = buildTool({
87      name: FILE_EDIT_TOOL_NAME,
88      searchHint: 'modify file contents in place',
89      maxResultSizeChars: 100_000,
90      strict: true,
91      async description() {
92        return 'A tool for editing files'
93      },
94      async prompt() {
95        return getEditToolDescription()
96      },
97      userFacingName,
98      getToolUseSummary,
99      getActivityDescription(input) {
100       const summary = getToolUseSummary(input)
101       return summary ? `Editing ${summary}` : 'Editing file'
102     },
103     get inputSchema() {
104       return inputSchema()
105     },
106     get outputSchema() {
107       return outputSchema()
108     },
109     toAutoClassifierInput(input) {
110       return `${input.file_path}: ${input.new_string}`
… 45 lines omitted; use the source link for the complete range …
156       }
157
158       // Check if path should be ignored based on permission settings
159       const appState = toolUseContext.getAppState()
160       const denyRule = matchingRuleForInput(
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	schema와 description
그 표현이 만든 조건	path permission
그 표현이 만든 조건	tool-specific input update
그 표현이 만든 조건	call의 별도 current read

언어와 런타임은 여기까지 보장한다

authority check와 optimistic stale check는 다른 실패 모델을 다룬다.

프로젝트가 이 메커니즘에 맡긴 책임

사용자 승인 뒤에도 call 시점 semantic validation이 필요하다.

경계를 잘못 읽을 때 생기는 위험

approval 결과를 그대로 성공 보증으로 간주하면 TOCTOU가 열린다.

이 설명을 확인할 코드 src/tools/FileEditTool/FileEditTool.ts:86-160

SELECTIVE EXECUTION TRACE

OBSERVED

사례: config.ts edit의 결정 장부

3권에서 넘어온 e1을 실제 effect로 닫는다.

책임 연결

Schema

path/old/new가 형식상 유효하다.

이때 남은 상태와 판단이 “Semantic”에서 사용할 전제가 된다.

책임 연결

Semantic

파일을 읽었고 old string이 현재 content에 존재한다.

이때 남은 상태와 판단이 “Permission”에서 사용할 전제가 된다.

판단 경계

Permission

최종 path와 input에 authority가 있다.

이때 남은 상태와 판단이 “Commit”에서 사용할 전제가 된다.

책임 연결

Commit

stale check 후 patch/write한다.

이때 남은 상태와 판단이 “Observation”에서 사용할 전제가 된다.

책임 연결

Observation

tool_result, history, LSP, transcript를 갱신한다.

이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

사용자가 본 preview, permission이 승인한 input, call이 실행한 input을 같은 digest로 연결한다.

어디에서 설명이 틀어지는가

write는 성공했지만 result append가 실패하면 5권의 recovery 문제가 된다.

이 설명을 확인할 코드 src/services/tools/toolExecution.ts:599-720 src/tools/FileEditTool/FileEditTool.ts:387-455

DISTINCTION TABLE

OBSERVED

Effect 결과의 네 상태

성공/실패 Boolean보다 복구에 필요한 상태를 구분한다.

상태	filesystem	protocol	다음 행동
not-started	변화 없음	거부/취소 result	재제안 가능
committed-observed	변경됨	result 기록됨	계속 진행
committed-unobserved	변경됨	result 누락	reconcile 후 재실행 금지
unknown	확인 필요	중단/오류	read-back 검증

이 표를 읽는 관점

외부 effect에서는 exactly-once보다 identity와 reconciliation이 현실적인 목표다.

CONCEPT IN CONTEXT

RECOMMENDATION

권장 분리: Effect·Policy·Presentation

Tool 인터페이스의 변경 이유를 분리하되 기존 registry compatibility를 유지한다.

코드에서 읽어낼 수 있는 단서

출발 단서	ToolEffect는 schema와 call을 소유한다.
연결되는 단서	ToolPolicy는 resource/effect classification과 permission을 소유한다.
연결되는 단서	ToolPresenter는 REPL component를 소유한다.
연결되는 단서	ToolDescriptor는 model-facing 설명을 소유한다.

이 단서들을 하나의 설계로 묶으면

기존 Tool facade는 조립 결과로 남겨 migration을 단계화할 수 있다.

시스템 이해에 남는 결론

먼저 contract test 없이 인터페이스만 분할하면 실행 순서와 UI 의미가 깨진다.

이 설명을 확인할 코드 src/Tool.ts:362-440

ALGORITHM A24 · 독자의 질문

Tool contract characterization 은 어떤 시스템 문제를 닫는가

거대한 Tool interface를 나누는 리팩터링은 파일 이동보다 기존 permission·default·rendering을 보존하는 일이 어렵다. snapshot이 통과해도 중요한 negative path를 놓칠 수 있다.

코드가 필요해지는 이유

거대한 interface를 나누면서 기존 default, permission, result rendering을 보존한다.

OBSERVED Tool factory와 registry의 현재 behavior가 characterization baseline을 제공한다.

INFERENCE 이 알고리즘은 제품 runtime이 아니라 구조 변경의 안전성을 만드는 verification 절차다.

이 절에서 답할 결정 질문

현재 behavior를 oracle로 만들되 잘못된 동작까지 무비판적으로 고정하지 않으려면 어떤 테스트 층이 필요한가?

> ### 이 코드를 읽기 전에 잡을 개념
>
> **characterization test와 contract test.** 문서화되지 않은 현재 동작을 먼저 관찰해 회귀를 찾고, 별도의 contract/negative test로 유지해야 할 의미를 선택한다.
>
> 743–791행의 defaults를 representative tool과 실제 permission context에서 다시 관찰한다.

이 설명을 확인할 코드 src/Tool.ts:743-791

ALGORITHM A24 · 실제 코드

RECOMMENDATION

Tool contract characterization 의 결정점은 어디에 있는가

리팩터링 전에 각 tool의 관찰 가능한 contract를 자동 추출·고정한다.

입력 표현. tool registry, schema samples, 그리고 permission contexts

보존되는 작업 상태. tool별 contract snapshot

factory가 보장하는 default

src/Tool.ts:743-791

```
743   /**
744    * Build a complete `Tool` from a partial definition, filling in safe defaults
745    * for the commonly-stubbed methods. All tool exports should go through this so
746    * that defaults live in one place and callers never need `?.() ?? default`.
747    *
748    * Defaults (fail-closed where it matters):
749    * - `isEnabled` → `true`
750    * - `isConcurrencySafe` → `false` (assume not safe)
751    * - `isReadOnly` → `false` (assume writes)
752    * - `isDestructive` → `false`
753    * - `checkPermissions` → `{ behavior: 'allow', updatedInput }` (defer to general permission system)
754    * - `toAutoClassifierInput` → `''` (skip classifier — security-relevant tools must override)
755    * - `userFacingName` → `name`
756    */
… 32 lines omitted; use the source link for the complete range …
789       userFacingName: () => def.name,
790       ...def,
791     } as BuiltTool<D>
```

> 이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A24 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

tool registry, schema samples, 그리고 permission contexts은 처음부터 신뢰된 실행 상태가 아니다. tool별 contract snapshot가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** enabled/name/schema를 snapshot한다. 이 결과가 다음 판단 "valid/invalid input corpus를 실행한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** valid/invalid input corpus를 실행한다. 이 결과가 다음 판단 "read-only/concurrency/destructive classification을 비교한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** read-only/concurrency/destructive classification을 비교한다. 이 결과가 다음 판단 "deny/ask/allow 결과를 고정한다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** deny/ask/allow 결과를 고정한다. 이 결과가 다음 판단 "result block과 UI projection을 비교한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** result block과 UI projection을 비교한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

실제 effect는 fake adapter로 격리한다. default behavior와 override를 모두 포함한다. snapshot 변경은 의미 변경으로 review한다.

ALGORITHM A24 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **검증·관측·운영의 질문으로 읽기.** 실제 effect는 fake adapter로 격리한다.
- **검증·관측·운영의 질문으로 읽기.** default behavior와 override를 모두 포함한다.
- **검증·관측·운영의 질문으로 읽기.** snapshot 변경은 의미 변경으로 review한다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- mock가 실제 OS semantics를 숨김
- feature variant 누락
- golden snapshot의 무비판 갱신

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 파일 이동보다 behavior preservation을 먼저 증명한다.

RECOMMENDATION property/negative test와 소수 real sandbox integration test를 함께 둔다.

ALGORITHM A24 · 운영과 현재 상태

작은 변환도 effect와 미확정 경계를 남긴다

type test, snapshot, contract, 소수 integration test가 서로 다른 failure를 검출한다.

실패 뒤 남는 상태

mock OS, variant 누락, 자동 snapshot 갱신은 behavior drift를 승인처럼 보이게 한다.

OBSERVED	factory가 보장하는 default와 tool contract가 source에 존재한다.
CONDITIONAL	representative tool과 feature variant를 포함한 테스트가 있을 때 interface 분해 회귀를 찾을 수 있다.
UNKNOWN	현재 test suite가 모든 contract를 덮는지는 선택 source로 확인되지 않는다.
NEXT READ	구조 이동 전 public behavior 목록과 risk-based negative sample을 먼저 확정해야 한다.

다시 적용할 질문

1. golden snapshot 변화가 의도된 개선인지 누락된 default인지 어떻게 구분할 것인가?
2. compile-only test와 real sandbox test가 각각 잡지 못하는 실패는 무엇인가?

THEORY ATLAS · 1/3

용어를 외우지 않고, 어떤 보장을 설명하는 이론인지 묶어 읽는다

이 지도는 이 권의 알고리즘이 실제로 참조한 이론만 집계한다. 빈도는 중요도 점수가 아니라 여러 구현 결정을 같은 질문으로 비교할 수 있는 탐색 신호다.

상태·계산 모델

4개 알고리즘 · 4개 표현

입력이 어떤 중간 표현과 상태 전이를 거쳐 결과가 되는지, 그리고 각 전이 전후에 무엇이 계속 참이어야 하는지를 설명한다.

이 권에서 나타나는 방식. A17 병렬 결과의 deterministic context merge과 A18 공통 permission-and-call pipeline에서 이 관점이 결정적인 이유는 "동시에 currentContext를 write하지 않는다. 또한 사용자가 승인한 input과 call input의 identity를 추적한다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. deterministic reduction(A17), pipeline(A18), short-circuit evaluation(A20), compositional analysis(A21)

스터디 질문. 단계의 순서나 표현을 바꿔도 같은 상태와 불변조건에 도달하는가?

동시성·시간·합의

4개 알고리즘 · 9개 표현

여러 실행이 같은 상태를 읽고 쓰는 동안 순서, 취소, 재시도, 관찰 시점이 결과의 의미를 바꾸지 않게 하는 조건을 다룬다.

이 권에서 나타나는 방식. A16 입력별 concurrency batch partition과 A17 병렬 결과의 deterministic context merge에서 이 관점이 결정적인 이유는 "parse 실패는 unsafe다. 또한 동시에 currentContext를 write하지 않는다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. commutativity(A16), conflict serializability(A16), conservative scheduling(A16), bounded concurrency(A17), optimistic concurrency(A22), version token(A22), optimistic concurrency control(A23), critical section(A23), compare-and-swap(A23)

스터디 질문. stale read, 중복 실행, 취소가 끼어들어도 effect와 최종 상태의 의미가 보존되는가?

프로토콜·식별

3개 알고리즘 · 4개 표현

서로 다른 계층이 같은 작업과 같은 메시지를 가리킨다는 사실을 타입, 식별자, completion 규약으로 유지한다.

이 권에서 나타나는 방식. A15 Generic factory의 default 합성과 A20 Tool-level deny/ask short circuit에서 이 관점이 결정적인 이유는 "def가 default를 덮어쓴다. 또한 deny는 하위 allow가 조용히 덮지 않는다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. structural typing(A15), conditional type(A15), namespace identity(A20), tagged union(A42)

스터디 질문. 생산자와 소비자가 동일한 값·작업·종료 의미를 해석한다는 증거가 있는가?

괄호 안의 알고리즘 ID로 돌아가면 이론의 정의가 아니라 그 이론이 설명하는 source transition, 반례, 불변조건을 함께 확인할 수 있다.

THEORY ATLAS · 2/3

용어를 외우지 않고, 어떤 보장을 설명하는 이론인지 묶어 읽는다

이 지도는 이 권의 알고리즘이 실제로 참조한 이론만 집계한다. 빈도는 중요도 점수가 아니라 여러 구현 결정을 같은 질문으로 비교할 수 있는 탐색 신호다.

내구성·복구

1개 알고리즘 · 1개 표현

부분 기록, 재시작, 버전 변화가 있어도 이전 사실을 잃거나 같은 effect를 다시 확정하지 않도록 만드는 복구 계약을 설명한다.

이 권에서 나타나는 방식. A17 병렬 결과의 deterministic context merge에서 이 관점이 결정적인 이유는 “동시에 currentContext를 write하지 않는다.”라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. two-phase merge(A17)

스터디 질문. 중단 지점에서 재개했을 때 무엇을 재생하고 무엇을 다시 실행하지 않아야 하는가?

권한·신뢰 경계

7개 알고리즘 · 12개 표현

실행 가능성, 허용 여부, 데이터 노출을 분리하고 신뢰하지 않은 입력이 effect 권한으로 곧바로 승격되지 않게 한다.

이 권에서 나타나는 방식. A18 공통 permission-and-call pipeline과 A19 Rule source aggregation과 behavior precedence에서 이 관점이 결정적인 이유는 “사용자가 승인한 input과 call input의 identity를 추적한다. 또한 rule에는 source가 남는다.”라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. reference monitor(A18), policy enforcement point(A18), policy combining algorithm(A19), provenance(A19), deny/permit precedence(A19), coarse/fine authorization(A20), meet/deny-overrides policy(A21), language sandboxing(A21), deny(A42), capability attenuation(A43), policy(A43), read set(A22)

스터디 질문. 누가 어떤 근거로 effect를 허용하며, 더 약한 권한으로만 위임된다는 사실을 어디서 확인하는가?

구성·변형·성능

2개 알고리즘 · 2개 표현

동일한 핵심 의미를 여러 실행 표면과 제품 변형에 조립하면서 비용과 의존성의 위치를 통제하는 방법을 다룬다.

이 권에서 나타나는 방식. A15 Generic factory의 default 합성과 A43 설정과 policy를 sandbox runtime capability로 투영에서 이 관점이 결정적인 이유는 “def가 default를 덮어쓴다. 또한 managed-only가 켜지면 낮은 authority source가 allow 범위를 넓히지 못한다.”라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. factory(A15), configuration composition(A43)

스터디 질문. 조립 방식이나 최적화를 바꿔도 핵심 계약이 adapter와 build 변형 안에서 보존되는가?

괄호 안의 알고리즘 ID로 돌아가면 이론의 정의가 아니라 그 이론이 설명하는 source transition, 반례, 불변조건을 함께 확인할 수 있다.

THEORY ATLAS · 3/3

용어를 외우지 않고, 어떤 보장을 설명하는 이론인지 묶어 읽는다

이 지도는 이 권의 알고리즘이 실제로 참조한 이론만 집계한다. 빈도는 중요도 점수가 아니라 여러 구현 결정을 같은 질문으로 비교할 수 있는 탐색 신호다.

검증·관측·운영

2개 알고리즘 · 4개 표현

실행 결과를 관찰하고 회귀를 검출하며, cleanup과 운영 신호가 본래 동작을 왜곡하지 않도록 만드는 증거 체계를 다룬다.

이 권에서 나타나는 방식. A42 Hook fan-out과 permission decision 합성과 A24 Tool contract characterization에서 이 관점이 결정적인 이유는 "신뢰되지 않은 workspace의 hook은 실행하지 않는다. 또한 실제 effect는 fake adapter로 격리한다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. fan-out(A42), characterization testing(A24), contract test(A24), golden master(A24)

스터디 질문. 최종 출력뿐 아니라 실패·중단·복구 경로의 보장을 어떤 관찰과 테스트로 입증할 수 있는가?

괄호 안의 알고리즘 ID로 돌아가면 이론의 정의가 아니라 그 이론이 설명하는 source transition, 반례, 불변조건을 함께 확인할 수 있다.

ALGORITHM INDEX

OBSERVED

이 권의 결정적 알고리즘과 보장

알고리즘 이름보다 마지막 열을 먼저 읽는다. 프로젝트에서 중요한 것은 계산 기법 자체보다 그 계산으로 어떤 시스템 조건을 계속 참으로 만드는가이기 때문이다.

ID	문제와 구현	대표 불변조건
A15	**Generic factory의 default 합성** 60개 이상의 tool이 반복 boilerplate 없이 complete Tool contract를 갖게 한다.	def가 default를 덮어쓴다.
A16	**입력별 concurrency batch partition** 읽기 계열 tool은 병렬화하고 effect 충돌 가능성이 있는 tool은 순서를 보존한다.	parse 실패는 unsafe다.
A17	**병렬 결과의 deterministic context merge** progress/result는 병렬로 내보내면서 shared ToolUseContext의 갱신 순서를 안정화한다.	동시에 currentContext를 write하지 않는다.
A18	**공통 permission-and-call pipeline** 모델이 만든 unknown JSON을 현재 workspace에서 실행 가능한 승인된 input으로 바꾼다.	사용자가 승인한 input과 call input의 identity를 추적한다.
A19	**Rule source aggregation과 behavior precedence** user/project/local/flag/policy/CLI/command/session rule을 provenance와 함께 effective decision에 사용한다.	rule에는 source가 남는다.
A20	**Tool-level deny/ask short circuit** 특정 tool 또는 MCP namespace 전체에 대한 정책을 input parsing과 별도로 적용한다.	deny는 하위 allow가 조용히 덮지 않는다.
A21	**Compound command decision 합성** pipeline/compound command 일부만 허용되거나 거부될 때 전체 command authority를 정한다.	부분 allow로 compound command를 실행하지 않는다.
A42	**Hook fan-out과 permission decision 합성** 서로 다른 source와 실행 방식(command, prompt, agent, HTTP, callback)의 hook을 한 lifecycle event에서 실행하면서 core effect의 최종 input과 authority를 결정한다.	신뢰되지 않은 workspace의 hook은 실행하지 않는다.
A43	**설정과 policy를 sandbox runtime capability로 투영** 여러 settings source의 filesystem/network 규칙을 OS별 sandbox runtime이 집행할 하나의 configuration으로 만든다.	managed-only가 켜지면 낮은 authority source가 allow 범위를 넓히지 못한다.
A22	**Read가 edit precondition을 만든다** 모델이 읽은 버전과 실제 편집 대상 버전이 같은지 나중에 판단할 근거를 만든다.	partial read 범위를 구분한다.
A23	**FileEdit optimistic validation과 commit window** 모델이 읽은 old string을 현재 파일에 안전하게 적용하고 외부 변경을 덮어쓰지 않는다.	check와 synchronous write 사이 내부 await가 없다.
A24	**Tool contract characterization** 거대한 interface를 나누면서 기존 default, permission, result rendering을 보존한다.	실제 effect는 fake adapter로 격리한다.

SOURCE EVIDENCE INDEX · 1/2

OBSERVED

설명을 다시 코드로 되돌리는 출발점

보고서의 해석은 아래 고정 줄 범위에서 다시 검토할 수 있다. 링크는 모두 동일한 commit을 가리키며, 파일 이름만 비슷한 최신 branch로 이동하지 않는다.

고정 코드 범위	연결되는 설명	표현 방식
src/Tool.ts:362-405	Tool generic은 입력과 출력 관계를 보존한다	code
src/Tool.ts:721-791	Generic factory의 default 합성	algorithm
src/services/tools/toolOrchestration.ts:84-116	입력별 concurrency batch partition	algorithm
src/services/tools/toolOrchestration.ts:19-81	병렬 결과의 deterministic context merge	algorithm
src/services/tools/toolExecution.ts:337-410	Unknown tool도 error result로 닫힌다	code
src/services/tools/toolExecution.ts:599-720	공통 permission-and-call pipeline	algorithm
src/tools.ts:1-150	getAllBaseTools는 정적 배열이 아니라 build·environment·product mode의 합성 결과다	code
src/tools.ts:151-260	getAllBaseTools는 정적 배열이 아니라 build·environment·product mode의 합성 결과다	code
src/types/permissions.ts:171-260	Permission result는 algebraic data type이다	code
src/utils/permissions/permissions.ts:109-131	Rule source aggregation과 behavior precedence	algorithm
src/utils/settings/constants.ts:3-22	Rule source aggregation과 behavior precedence	algorithm
src/utils/permissions/permissions.ts:1070-1120	Tool-level deny/ask short circuit	algorithm
src/tools/BashTool/bashPermissions.ts:1185-1225	Shell command는 문자열이 아니라 외부 언어다	code
src/tools/BashTool/bashPermissions.ts:2430-2485	Compound command decision 합성	algorithm
src/utils/permissions/filesystem.ts:1-80	Path는 문자열이 아니라 시간에 따라 변하는 resource identity다	explain
src/utils/hooks/hooksConfigManager.ts:26-70	Hook event 이름은 단순 notification이 아니라 서로 다른 제어권을 뜻한다	code
src/utils/hooks.ts:1952-2075	Hook fan-out과 permission decision 합성	algorithm
src/utils/hooks.ts:2445-2685	Hook fan-out과 permission decision 합성	algorithm
src/utils/hooks.ts:2760-2895	Hook fan-out과 permission decision 합성	algorithm
src/utils/hooks.ts:520-670	Hook output은 JSON parse 뒤에도 event별 재검증을 거친다	code
src/utils/hooks/execHttpHook.ts:52-180	HTTP hook는 URL뿐 아니라 DNS·proxy·header secret 경계를 함께 검증한다	code
src/utils/sandbox/sandbox-adapter.ts:172-260	설정과 policy를 sandbox runtime capability로 투영	algorithm
src/utils/sandbox/sandbox-adapter.ts:704-799	설정과 policy를 sandbox runtime capability로 투영	algorithm
src/tools/BashTool/shouldUseSandbox.ts:123-154	설정과 policy를 sandbox runtime capability로 투영	algorithm

SOURCE EVIDENCE INDEX · 2/2

OBSERVED

설명을 다시 코드로 되돌리는 출발점

보고서의 해석은 아래 고정 줄 범위에서 다시 검토할 수 있다. 링크는 모두 동일한 commit을 가리키며, 파일 이름만 비슷한 최신 branch로 이동하지 않는다.

고정 코드 범위	연결되는 설명	표현 방식
src/tools/BashTool/shouldUseSandbox.ts:1-82	excludedCommands는 security boundary가 아니라 sandbox 선택 heuristic이다	code
src/utils/hooks.ts:3390-3485	Bash tool 하나가 hook와 sandbox를 모두 지나는 경우	walkthrough
src/tools/FileReadTool/FileReadTool.ts:1020-1040	Read가 edit precondition을 만든다	algorithm
src/tools/FileEditTool/FileEditTool.ts:427-491	FileEdit optimistic validation과 commit window	algorithm
src/tools/FileEditTool/FileEditTool.ts:599-625	FileEdit optimistic validation과 commit window	algorithm
src/tools/FileEditTool/FileEditTool.ts:86-160	checkPermissions와 call은 서로 다른 시점의 world를 본다	code
src/tools/FileEditTool/FileEditTool.ts:387-455	사례: config.ts edit의 결정 장부	walkthrough
src/Tool.ts:362-440	권장 분리: Effect·Policy·Presentation	explain
src/Tool.ts:743-791	Tool contract characterization	algorithm

STUDY SYNTHESIS

Effect contract 검증

읽은 내용을 요약하는 대신, 새로운 변경이나 장애 상황에서도 같은 보장을 찾아낼 수 있는지 확인한다.

검증 질문	Tool generic과 Zod schema의 역할을 나눈다.
검증 질문	partitionToolCalls의 fail-conservative branch를 찾는다.
검증 질문	PreToolUse hook mutation과 permission precedence에서 최종 input identity를 기록한다.
검증 질문	sandbox routing과 OS capability가 permission allow와 다른 보장임을 설명한다.
검증 질문	Edit e1의 commit point와 committed-unobserved recovery를 설계한다.

이 권의 완료 기준

"도구를 실행한다"를 parse·hook mutation·authorize·sandbox·commit·observe의 보존 가능한 contract로 설명한다.

답에는 반드시 현재 snapshot의 코드 근거, 그 근거에서 도출한 해석, 아직 확인할 수 없는 부분을 구분해 적는다.